

COMP 3311 数据库管理系统 — 完整课程笔记

课程: COMP 3311 Database Management Systems

学期: Summer 2026

覆盖: Lecture 1 – Lecture 24 (全部课程内容)

教材: "Fundamentals of Database Systems"

总目录

Part 1: Lectures 1–7 — 概念建模、关系设计、范式化、关系代数

- - 1. Lecture 1: 数据库管理系统概述
- - 2. Lecture 2: E-R 数据模型 (上)
- - 3. Lecture 3: E-R 数据模型 (下) — 约束与设计选择
- - 4. Lecture 4: 关系数据库设计 — E-R 到关系模式的规约
- - 5. Lecture 5: 关系数据库设计 — 函数依赖
- - 6. Lecture 6: 关系数据库设计 — 范式化
- - 7. Lecture 7: 关系代数

Part 2: Lectures 8–11 — SQL DML & DDL

- - 8. Lecture 8: SQL DML — 基本查询结构
- - 9. Lecture 9: SQL DML — 聚合与子查询
- - 10. Lecture 10: SQL DML — 分析函数与数据修改
- - 11. Lecture 11: SQL DML & DDL — 过程化 SQL 与模式定义

Part 3: Lectures 12–15 — NoSQL & MongoDB

-
- 12. Lecture 12: NoSQL 数据库管理系统
-
- 13. Lecture 13: MongoDB — 数据模型与查询语言
-
- 14. Lecture 14: MongoDB — 数组查询与模式验证
-
- 15. Lecture 15: MongoDB — 聚合框架

Part 4: Lectures 16–24 — 存储、索引、查询处理、事务与恢复

-
- 16. Lecture 16: 存储与文件结构
-
- 17. Lecture 17: 索引 — 基础概念与有序索引
-
- 18. Lecture 18: 索引 — B+-树索引与哈希索引
-
- 19. Lecture 19: 查询处理 — 成本估算
-
- 20. Lecture 20: 查询处理 — 连接操作与表达式求值
-
- 21. Lecture 21: 查询处理 — 大小估算与查询优化
-
- 22. Lecture 22: 事务管理 — 事务与可串行化
-
- 23. Lecture 23: 事务管理 — 并发控制协议
-
- 24. Lecture 24: 事务管理 — 恢复系统与 NoSQL 事务

附录

- 课程知识体系总图
- 参考模式速查
- MongoDB 聚合框架 — 模式速查

Part 1: Lectures 1–7 — 概念建模、关系设计、范式化、关系代数

1.1 Lecture 1: 数据库管理系统概述

1.1.1 核心概念

术语	定义
数据库 (Database)	有组织地收集的大量相关数据，具有特定目的、反应真实世界、包含语义信息
DBMS	通用软件系统，用于定义、存储、操作、共享和保护数据库
数据模型 (Data Model)	描述数据组织和访问方式的语言，包含：数据结构类型、完整性约束、操作
模式 (Schema)	用数据模型描述的数据库结构（相对稳定）
实例 (Instance)	数据库中某个时间点的实际数据内容（频繁变化）

1.1.2 为什么需要 DBMS？（文件系统的 7 大缺陷）

问题	说明
数据重复与不一致	同一数据存于多处，容易不一致
难以满足未预见需求	数据结构专为特定应用定制
相关数据隔离	关联数据散落在不同文件和格式中
约束管理困难	约束嵌入程序代码中，难以修改和执行
更新缺乏原子性	系统故障可能导致数据不一致（如转账）
并发访问问题	多用户同时访问导致数据丢失（如 lost update）
安全性难以保障	临时性数据管理导致安全策略难以实施

1.1.3 DBMS 的核心原则

- 集成统一组织数据
- 将元数据（系统目录）与原始数据分离
- 支持多用户
- 集中控制数据定义和访问

1.1.4 常见数据模型对比

模型	数据组织	关系表示	是否使用模式
E-R	实体与关系	显式（图中表示）	是
关系模型	表（Table）	隐式（数据中）	是
Key-Value	键-值对	聚合在数据中	否
文档模型	文档（如JSON）	聚合在数据中	否

1.1.5 三级数据抽象（仅关系型 DBMS）

View Level（视图层）－ 为特定应用提供数据子集，可隐藏/添加信息

↑ **Logical Data Independence**

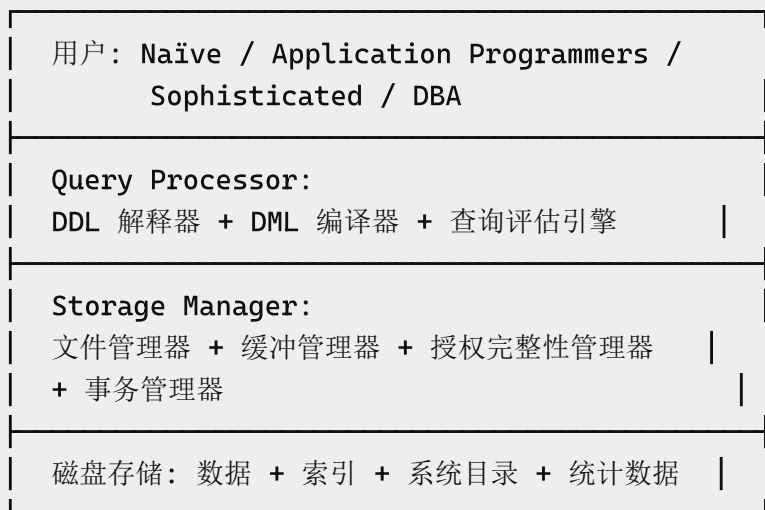
Logical Level（逻辑层）－ 描述存什么数据及其关系

↑ **Physical Data Independence**

Physical Level（物理层）－ 描述数据如何存储在磁盘上

- **逻辑数据独立性**：修改逻辑模式不影响视图模式
- **物理数据独立性**：修改物理模式不影响逻辑模式

1.1.6 DBMS 总体架构



- **DDL**（数据定义语言）：定义数据库模式
- **DML**（数据操作语言）：访问和操作数据
 - **过程式 DML**：指定需要什么数据以及如何获取（如 MQL）
 - **声明式 DML**：只指定需要什么数据（如 SQL）

1.1.7 用户类型

用户类型	描述
Naïve Users	使用现有应用程序（如打印报表）
Application Programmers	使用 DML 开发访问 DBMS 的应用程序
Sophisticated Users	直接使用查询语言或分析工具
DBA	协调 DBMS 的所有活动（模式、物理组织、性能、权限）

1.2 Lecture 2: E-R 数据模型（上）

1.2.1 数据库设计流程

需求分析 → 概念模式(ER图) → 逻辑模式(DDL) → 物理模式 → 实施

- **需求分析**：理解应用领域，识别数据需求
- **逻辑数据库设计**：概念模式（E-R 图，DBMS 无关）→ 逻辑模式（DDL，DBMS 相关）
- **物理数据库设计**：描述数据如何存储在存储介质上

1.2.2 E-R 模型三大基本概念

实体 (Entity)

- 应用中我们想存储数据的**事物**（如 Employee, Student, Course）
- 实体类型：对所有实体实例的公共描述
- 实体集：所有实体实例的集合
- 每个实体实例**具有唯一标识**（identity）

属性 (Attribute)

- 实体的**属性**，描述该属性的数据值
- 属性类型：
 - **简单/单值属性** — 基本属性
 - **复合属性** — 可分解的（如 *address* → *streetNo*, *streetName*）
 - **多值属性** — 用 { } 标记（如 {*skill*}）
 - **派生属性** — 可从其他属性计算得出，用 () 标记（如 (*age*) 从 *birthdate* 派生）
- 属性值可以为 **null**（缺失、未知或不适用）

关系 (Relationship)

- 实体之间的**关联描述**
- 关系本质上是**双向的**
- 关系可以有自己的属性（如 WorksOn 中的 `%time`）
- 关系度（Degree）：
 - **一元 (unary)**：同一实体与自己关联
 - **二元 (binary)**：两个不同实体关联（最常见）
 - **三元 (ternary)**：三个实体关联（极少见）
- **角色名 (Role Name)**：用于一元关系，标识实体在关系中扮演的角色

1.2.3 泛化/特化 (Generalization/Specialization)

- **超类型 (Supertype)** → 共性属性
- **子类型 (Subtype)** → 特有属性
- 分类方式：
 - **用户定义 (User-defined)**：由设计者指定子类型归属
 - **谓词定义 (Predicate-defined)**：由鉴别器属性（discriminator）值决定

1.2.4 继承 (Inheritance)

- 子类型从超类型**继承**所有属性和关系
- 子类型可**添加**新的属性/关系
- 设计指南：继承不应超过 2–3 层
- **多重继承**：子类型从多个超类型继承，可能导致命名冲突（通过重命名解决）

1.3 Lecture 3: E-R 数据模型（下）— 约束与设计选择

1.3.1 约束概述

约束是对数据的**逻辑限制**，任何数据都必须满足。约束为数据添加额外的语义。

重要认识：不是所有约束都能/应该在 E-R 模型中表示——有些留给应用程序处理更合适。

1.3.2 属性约束

域约束 (Domain Constraint)

- 限制属性的取值类型和范围
- 可通过**数据类型**（如 integer, char(20)）或**谓词**（如 salary 在 0–100,000）来指定

键约束 (Key Constraint)

- **候选键 (Candidate Key)**: 最小属性集, 能唯一标识一个实体实例
- **主键 (Primary Key)**: 从候选键中选出的一个, DBMS 自动强制唯一性
- **复合键 (Composite Key)**: 由多个属性组成
- 其他候选键的唯一性**不会被** DBMS 自动强制

1.3.3 强实体 vs 弱实体

特性	强实体 (Strong Entity)	弱实体 (Weak Entity)
主键	有	无
存在依赖	独立存在	依赖标识实体
标识方式	自有主键	标识实体主键 + 鉴别器 (如有)
E-R 图中关系线	虚线	实线 (标识关系)

1.3.4 泛化约束: 覆盖 (Coverage)

不相交性 (Disjointness)

- **不相交 (disjoint)**: 超类型实例最多属于一个子类型
- **重叠 (overlapping)**: 超类型实例可属于多个子类型

完备性 (Completeness)

- **完全 (total)**: 超类型实例**必须**属于至少一个子类型
- **部分 (partial)**: 超类型实例**不必须**属于任何子类型

常见组合: disjoint,total | disjoint,partial | overlapping,total | overlapping,partial

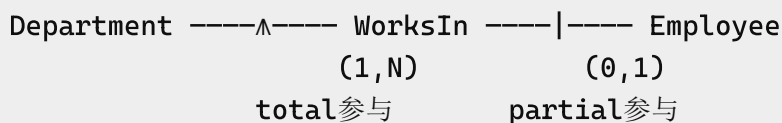
1.3.5 关系约束: 基数与参与

基数约束 (Cardinality) = 最大参与数:

- `max-card = 1`: 一对一 (1:1) 或一对多 (1:N)
- `max-card = N`: 多对多 (N:M)

参与约束 (Participation) = 最小参与数:

- `min-card = 0` : **部分参与** (partial) — 不必须
- `min-card $\ge$ 1` : **完全参与** (total) — 必须



1.3.6 排斥约束 (Exclusion/XOR)

实体实例在多个关系中**最多只能参与一个**（不能同时属于两个）。例如：Task 要么属于 InternalProject **要么**属于 ExternalProject，不能两者都是。

1.3.7 设计选择

对比	何时选择
实体 vs 属性	当概念有多个属性/关系时用实体；结构简单时用属性
强实体 vs 弱实体	有唯一标识时用强实体；没有唯一标识时用弱实体
实体 vs 关系	当概念代表应用域中独立的事物时用实体；当两个实体之间的关联仅为单一事实时用关系
属性放在哪	属性依赖一个实体 → 放该实体；属性仅当关系存在时才存在 → 放关系上

关键限制：在同一个关系类型中，两个实体实例之间最多只能有一条关系实例 → 这就是为什么一个人在同一支行开两个账户时需要把 Account 做成实体。

1.3.8 Crow Foot 符号汇总

概念	符号
实体	矩形框
属性	列在实体框内
主键	下划线
多值属性	{ }
派生属性	()
1:1 关系	`——
1:N 关系	`—— 
N:M 关系	 —— 
标识关系 (弱实体)	实线 

概念	符号
排斥约束	X 符号 + 多关系
泛化/特化	三角+圆圈符号

1.4 Lecture 4: 关系数据库设计 — E-R 到关系模式的规约

1.4.1 关系数据模型基础

概念	说明
关系 (Relation)	表, 记为 $R(A_1, A_2, \dots, A_n)$
属性 (Attribute)	列
域 (Domain)	属性的类型和取值范围
元组 (Tuple)	行
度 (Degree)	属性的数量
势 (Cardinality)	元组的数量

关系的性质:

- 元组是**无序的** (关系是集合)
- 所有属性值必须是**原子的** (不允许复合/多值属性)
- 关系实例是域的笛卡尔积的**任意子集**: $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$

1.4.2 键 (Keys)

Superkey (超键) → 任何能唯一标识元组的属性集



Candidate Key (候选键) → 最小的超键 (不能再去掉任何属性)



Primary Key (主键) → 设计者选出的一个候选键

- **实体完整性约束**: 主键不能包含 null 值
- **参照完整性约束** (外键 Foreign Key): $T.fk_s$ 的值要么等于 S 中某元组的主键值, 要么全部为 null
- 外键表示 1:1 或 1:N 关系

1.4.3 E-R 模式到关系模式的规约规则

强实体

$E(k_s, \dots) \rightarrow R_s(k_s, \dots)$

弱实体

T (弱实体, 鉴别器 d_t) 依赖 S (强实体, 主键 k_s)
 $\rightarrow R_t(fk_s, d_t, a_r, \dots)$
PK = (fk_s, d_t)
FK: fk_s REFERENCES S(k_s) ON DELETE CASCADE

复合属性

- 选项 1: 拼接为一个属性
- 选项 2: 拆分为多个独立属性

多值属性

$S(k_s, \dots, \{m\})$
 $\rightarrow S(k_s, \dots) + SM(fk_s, m)$ (PK = 全部属性)
FK: fk_s REFERENCES S(k_s) ON DELETE CASCADE

泛化/特化

- **选项 1** (通用): 每个实体一个关系模式。子类型添加父类型主键作为外键+主键, ON DELETE CASCADE。
- **选项 2** (仅 total, disjoint 可用): 只创建子类型关系模式, 包含父类型全部属性。

二元关系

$R(a_r, \dots)$ between $S(k_s, \dots)$ and $T(k_t, \dots)$
 $1:1 \rightarrow R_r(fk_s, fk_t, a_r)$ PK = fk_s 或 fk_t
 $1:N \rightarrow R_r(fk_s, fk_t, a_r)$ PK = fk_t (N 侧)
 $N:M \rightarrow R_r(fk_s, fk_t, a_r)$ PK = (fk_s, fk_t)

模式合并 (优化)

- 1:1: 关系模式可与任一方实体合并

- 1:N：关系模式可与 N 侧实体合并
- 合并后外键的参照完整性动作由参与约束决定：
 - partial → ON DELETE SET NULL
 - total → ON DELETE CASCADE

排斥约束

- 选项 1：单一关系，两个外键 + CHECK 约束确保只有一个非 null
- 选项 2：单一关系，合并外键 + 类型判别属性（仅当键域相同时可用）

1.4.4 代理键 (Surrogate Key)

- 新增的无应用意义的属性，用作主键
- 用途：将弱实体变为强实体，或替代过长的复合主键
- **绝不引入到 E-R 模式中**，只在规约到关系模式时添加

1.5 Lecture 5: 关系数据库设计 — 函数依赖

1.5.1 函数依赖 (FD) 定义

在关系模式 R 中， $X \rightarrow Y$ 意味着：**任何时候**，给定 X 的值，最多只有一个 Y 的值与之对应。

- **平凡 FD (Trivial FD)**： $Y \subseteq X$ （永远成立）
- **非平凡 FD (Nontrivial FD)**： $Y \cap X = \emptyset$ （作为约束，约束合法的关系实例）

1.5.2 Armstrong 公理系统

规则	名称	含义
IR1	自反律 (Reflexivity)	如果 $Y \subseteq X$ ，则 $X \rightarrow Y$
IR2	增广律 (Augmentation)	$X \rightarrow Y \models XZ \rightarrow YZ$
IR3	传递律 (Transitivity)	$X \rightarrow Y, Y \rightarrow Z \models X \rightarrow Z$

附加规则（可由 IR1-IR3 推导）：

| IR4 | 合并律 (Union) | $X \rightarrow Y, X \rightarrow Z \models X \rightarrow YZ$ |

| IR5 | 分解律 (Decomposition) | $X \rightarrow YZ \models X \rightarrow Y$ 和 $X \rightarrow Z$ |

| IR6 | 伪传递律 (Pseudo-transitivity) | $X \rightarrow Y, WY \rightarrow Z \models WX \rightarrow Z$ |

- Armstrong 公理是**可靠 (sound)** 且**完备 (complete)** 的
- **F 的闭包 F^+** : 从 F 推出的所有 FD 的集合

1.5.3 属性闭包 (Attribute Closure)

X^+ (X 在 F 下的闭包) 是由 X 在函数依赖 F 下能确定的所有属性的集合。

$X \rightarrow Y$ 属于 F^+ **当且仅当** $Y \subseteq X^+$

计算算法:

```
 $X^{(0)} = X$ 
Repeat:
     $X^{(i+1)} = X^{(i)} \cup Z$ 
    (where  $\exists Y \rightarrow Z$  in F and  $Y \subseteq X^{(i)}$ )
Until  $X^{(i+1)} = X^{(i)}$ 
Return  $X^{(i+1)}$ 
```

属性闭包的用途:

1. 测试超键: X^+ 包含 R 的所有属性 $\rightarrow X$ 是超键; 如果 X 最小 $\rightarrow X$ 是候选键
2. 测试 FD: 要检验 $X \rightarrow Y$ 是否在 F^+ 中, 计算 X^+ 并检查 $Y \subseteq X^+$
3. 计算 F^+ : 对每个 $X \subseteq R$, 计算 X^+ , 输出 $X \rightarrow Y$ (对所有 $Y \subseteq X^+$)

素属性 (Prime Attribute): 属于某个候选键的属性; 否则为非素属性。

1.5.4 规范覆盖 (Canonical Cover) F_c

- F_c 与 F 等价 (拥有相同的 F^+)
- F_c 无冗余属性
- F_c 中每个 FD 的 LHS 唯一

计算算法:

1. $F_c = F$
2. 重复:
 - 使用合并律 (IR4) 合并 LHS 相同的 FD
 - 找到并删除无关属性 (extraneous attributes)

3. 直到 Fc 不再变化

示例： $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\} \rightarrow Fc = \{A \rightarrow B, B \rightarrow C\}$

1.6 Lecture 6: 关系数据库设计 — 范式化

1.6.1 范式化目标

范式化利用函数依赖将**不满足要求**的关系模式**分解**为更理想的关系模式。

设计五大指南：

指南	内容
Guideline 1	清晰的属性语义 — 不要把多个实体类型混在一个关系模式中
Guideline 2	最小化 null 值 — 避免将有大量 null 的属性放在同一模式中
Guideline 3	最小化冗余 （消除操作异常） — 插入/删除/更新异常
Guideline 4	无损分解 (Lossless Join) — 连接后能恢复原关系
Guideline 5	保持函数依赖 — FD 不要分散在多个关系模式中（避免 join 才能验证约束）

无损分解条件 ($R \rightarrow R1, R2$): $R1 \cap R2 \rightarrow R1$ 或 $R1 \cap R2 \rightarrow R2$ 在 F^+ 中。
(公共属性必须是 $R1$ 或 $R2$ 的超键。)

1.6.2 各级范式总结

1NF $\supset$ 2NF $\supset$ 3NF $\supset$ BCNF
(越往后要求越严格)

第一范式 (1NF)

- 所有属性值都是**原子**的（无复合、无多值）
- 关系模型的固有属性，规约出的关系模式永远满足 1NF

第二范式 (2NF)

所有**非素属性**对**每个候选键**都是**完全函数依赖**的。

- 消除**部分依赖**：候选键的真子集 $\rightarrow$ 非素属性

示例（违反 2NF）：

Car(make, model, engineSize, fee, origin, tax)

PK: (make, model, engineSize)

违反 FD: $\text{engineSize} \rightarrow \text{fee} \leftarrow \text{engineSize}$ 是 PK 的真子集, fee 是非素属性

修复：分解为 Licensing(engineSize, fee) 和 Car(make, model, engineSize, origin, tax)。

第三范式 (3NF)

在 2NF 基础上, 所有**非素属性**对每个候选键都是**非传递依赖**的。

即：对于任意 $X \rightarrow A$ 在 F^+ 中, 至少满足其一：

1. $A \in X$ (平凡 FD)
2. X 是超键 (superkey)
3. A 是素属性 (prime attribute)

- 消除**传递依赖**：非超键 $\rightarrow$ 非素属性

示例（违反 3NF）：

Car(make, model, engineSize, origin, tax)

违反 FD: $\text{origin} \rightarrow \text{tax} \leftarrow \text{origin}$ 不是超键, tax 是非素属性

3NF 分解（合成）算法

计算 F_c （规范覆盖）

$S = \emptyset$

对每个 FD $X \rightarrow Y$ 在 F_c 中：

$S = S \cup (X, Y)$

如果 S 中没有任何模式包含 R 的候选键：

$S = S \cup K$ （添加一个候选键）

- **总是**产生无损连接、依赖保持的 3NF 分解。

Boyce-Codd 范式 (BCNF)

每个 FD 的**行列式 (LHS)** 都是超键。

即：对于任意 $X \rightarrow A$ 在 F^+ 中，至少满足其一：

1. $A \in X$ (平凡 FD)

2. X 是超键

- 比 3NF 更严格：3NF 允许"A 是素属性"的例外，BCNF 不允许
- BCNF = **无冗余** (可由 FD 消除的冗余)
- BCNF **不一定是** 依赖保持的！

示例：

Car(make, model, engineSize, origin)

违反 FD: $\text{origin} \rightarrow \text{engineSize}$ $\leftarrow$ origin 不是超键

分解 $\rightarrow$ Car(make, model, origin) + Country(origin, engineSize)

但这丢失了 FD: $\text{make, model, engineSize} \rightarrow \text{origin}$ **✗** 不保持依赖！

BCNF 分解算法

$S = \{R\}$

直到 S 中所有模式都在 BCNF 中：

对违反 BCNF 的 $X \rightarrow Y$ ：

$S = (S - R) \cup (R - Y) \cup (X, Y)$

- 总是产生**无损连接**的分解
- 但**不能保证依赖保持**

1.6.3 范式选择指南

范式	优点	缺点
3NF	总是存在依赖保持的分解	可能包含一些冗余
BCNF	消除更多冗余	可能不存在依赖保持的分解

实用指导：实践中，3NF 对大多数应用来说"够好了"。

1.7 Lecture 7: 关系代数

1.7.1 关系查询语言

类型	描述	代表
关系代数 (RA)	过程式 — 描述如何计算	基本操作 + 附加操作
关系演算 (RC)	声明式 — 只描述要什么	如 SQL 的基础

闭包性质：关系代数的每次操作输入一个/多个关系，输出也是关系 → 操作可以**组合**（嵌套）。

1.7.2 基本操作

操作	符号	说明
选择 (Selection)	$\sigma_C(R)$	选择满足条件 C 的元组（行）；模式不变
投影 (Projection)	$\pi_L(R)$	保留列表 L 中的属性（列）； 自动去重
并 (Union)	$R_1 \cup R_2$	属于 R_1 或 R_2 的元组；要求 并兼容 （同属性数+同类型）
差 (Set Difference)	$R_1 - R_2$	属于 R_1 但不属于 R_2 的元组
笛卡尔积 (Cartesian Product)	$R_1 \times R_2$	将 R_1 每个元组与 R_2 每个元组组合（巨大结果）

1.7.3 附加操作

操作	符号	说明
交 (Intersection)	$R_1 \cap R_2$	同时属于两者的元组；可由差表示
θ -连接 (θ -Join)	$R_1 \bowtie_C R_2$	笛卡尔积 + 选择 = $\sigma_C(R_1 \times R_2)$
等值连接 (Equijoin)	—	连接条件全为等值
自然连接 (Natural Join)	$R_1 \bowtie R_2$	基于同名属性等值的等值连接，结果只保留一个共同属性副本
左外连接 (Left Outer Join)	$R_1 \text{ \textbackslash leftouterjoin } R_2$	自然连接 + R_1 中不匹配的元组（填充 null）

操作	符号	说明
右外连接 (Right Outer Join)	$R_1 \searrow \text{rightouterjoin } R_2$	自然连接 + R_2 中不匹配的元组 (填充 null)
全外连接 (Full Outer Join)	$R_1 \searrow \text{fullouterjoin } R_2$	自然连接 + 双方不匹配的元组 (填充 null)
更名 (Rename)	$\rho_x(E)$	给表达式结果重命名

1.7.4 操作总结

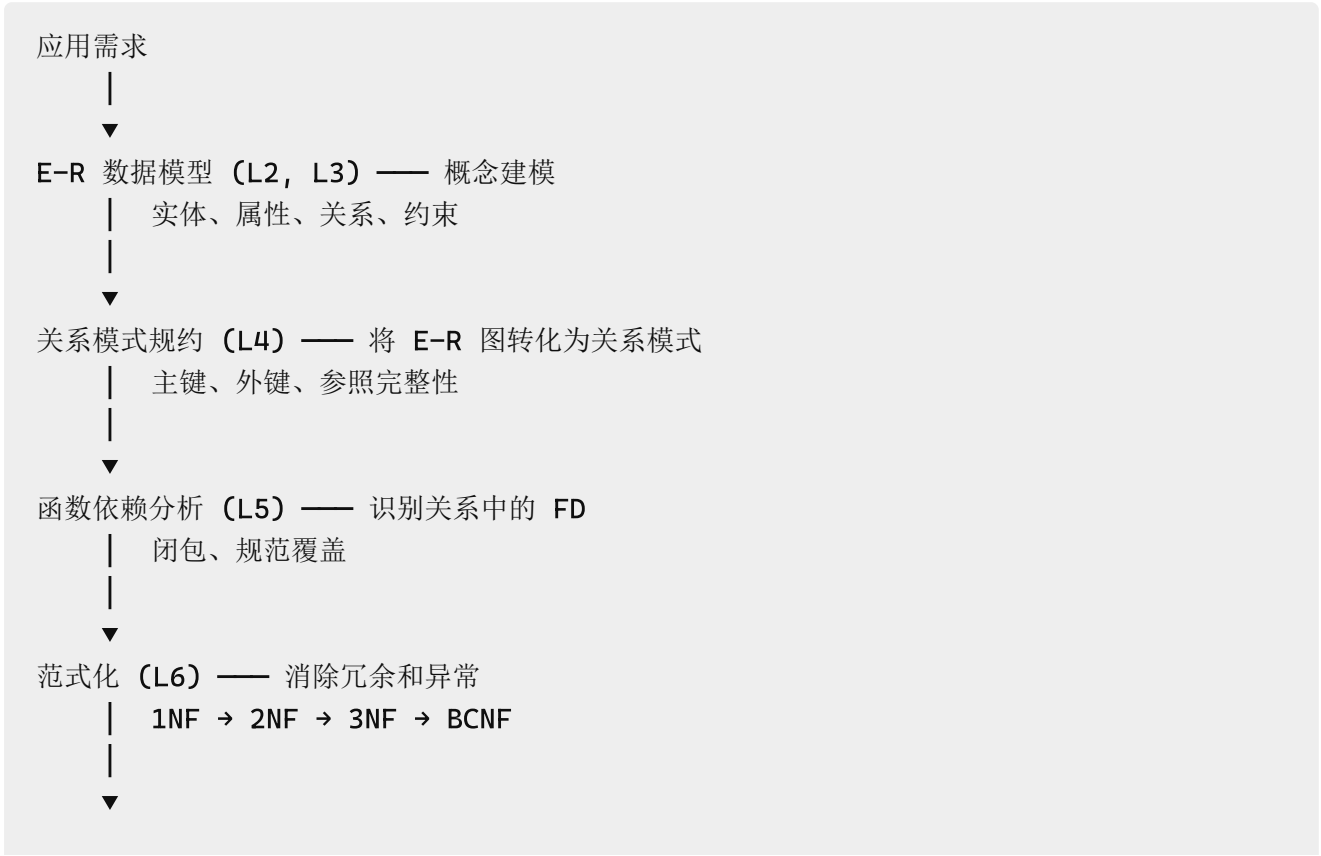
基础操作（不能由其他操作表示）：

- σ 选择 (一元)
- π 投影 (一元)
- $\cup$ 并 (二元, union-compatible)
- $-$ 差 (二元, union-compatible)
- $\times$ 笛卡尔积 (二元)

附加操作（可由基础操作表示）：

- $\cap$ 交 = $((R \cup S) - (R - S)) - (S - R)$
- $\bowtie$ 连接 = $\sigma_C(R \times S)$
- $\bowtie \bowtie \bowtie$ 外连接 = 自然连接 + null填充
- ρ 更名
- $\leftarrow$ 赋值

1.7.5 Part 1 核心概念关联图



关系代数 (L7) —— 查询关系数据库

| $\sigma, \pi, \cup, -, \times, \bowtie, \bowtie\bowtie\bowtie$
|
▼

SQL (L8+) —— 下一阶段!

Part 2: Lectures 8–11 — SQL DML & DDL

时间: 2026年6月20日–26日

覆盖: Lecture 8 – Lecture 11 (SQL DML & DDL)

参考数据库模式 (Bank Schema):

```
Account(accountNo, balance, branchName)
Borrower(clientId, loanNo)
Branch(branchName, district, liabilities, assets)
Client(clientId, name, hkid, address, district, rating)
Depositor(clientId, accountNo)
Loan(loanNo, amount, year, branchName)
Tags(clientId, tag)
```

2.8 Lecture 8: SQL DML — 基本查询结构

2.8.1 SQL 概述

SQL 由两部分组成:

- **DDL** (数据定义语言): 定义关系模式
- **DML** (数据操作语言): 查询和修改数据

SQL 查询的基本形式对应关系代数 $\pi_{A_1, A_2, \dots, A_n}(\sigma_{P_1}(\sigma_{P_2}(\dots \sigma_{P_m}(R_1 \times R_2 \times \dots \times R_m)))$:

```
SELECT A1, A2, ..., An
FROM R1, R2, ..., Rm
WHERE P;
```

关键性质: SQL 查询结果是关系 (但**可能包含重复**)，所以查询可以组合/嵌套。

2.8.2 SELECT 子句 (投影)

功能	语法	说明
基本投影	<code>SELECT \$a_1\$, \$a_2\$ FROM R</code>	选择指定列
所有属性	<code>SELECT * FROM R</code>	返回所有列
去重	<code>SELECT DISTINCT a FROM R</code>	默认不去重 ，需显式使用 DISTINCT
保留重复	<code>SELECT ALL a FROM R</code>	等同于省略 ALL（默认行为）
算术运算	<code>SELECT a*100 FROM R</code>	可包含 +, -, *, /
空值替换	<code>SELECT COALESCE(a, 'unknown')</code>	返回第一个非 null 参数

2.8.3 WHERE 子句（选择）

比较运算符

= 等于	> 大于	< 小于
>= 大于等于	<= 小于等于	<> 或 != 不等于

特殊运算符

运算符	语法	说明
BETWEEN	<code>WHERE rating BETWEEN 7 AND 9</code>	范围查询（包含端点）
NOT BETWEEN	<code>WHERE rating NOT BETWEEN 7 AND 9</code>	范围排除
AND/OR/NOT	<code>WHERE a=1 AND b=2</code>	布尔运算（AND 优先级高于 OR）
IS NULL	<code>WHERE district IS NULL</code>	不能用 = NULL （null 不匹配任何值）
IS NOT NULL	<code>WHERE district IS NOT NULL</code>	非空判断
LIKE	<code>WHERE addr LIKE '%Main%'</code>	字符串模式匹配： <code>%</code> = 任意子串， <code>_</code> = 单个字符
ESCAPE	<code>WHERE s LIKE '20\%%' ESCAPE '\'</code>	转义特殊字符
REGEXP_LIKE	<code>WHERE REGEXP_LIKE(name, '^Ste(v ph)en')</code>	正则表达式匹配（Oracle）

重要： null 比较的陷阱 — `WHERE district = null` **永远不会** 匹配任何行！

2.8.4 FROM 子句（连接）

连接类型	SQL 语法	说明
笛卡尔积	FROM R_1 , R_2 或 FROM R_1 CROSS JOIN R_2	所有组合
自然连接	FROM R_1 NATURAL JOIN R_2	所有同名属性等值连接，公共属性只出现一次
USING 连接	FROM R_1 JOIN R_2 USING (a)	指定公共属性连接
θ -连接 / 等值连接	FROM R_1 JOIN R_2 ON $R_1.a = R_2.b$	用 ON 指定任意连接条件
WHERE 连接	FROM R_1 , R_2 WHERE $R_1.a = R_2.b$	等价于等值连接（传统写法）
左外连接	FROM R_1 LEFT OUTER JOIN R_2 ON ...	R_1 中不匹配的元组保留（填充 null）
右外连接	FROM R_1 RIGHT OUTER JOIN R_2 ON ...	R_2 中不匹配的元组保留
全外连接	FROM R_1 FULL OUTER JOIN R_2 ON ...	双方不匹配的都保留

注意事项：

- 自然连接不能用 NATURAL + USING — 两者互斥
- 自然连接中**不能**用表名限定公共属性（如 $R.a$ ）
- 外连接的连接条件**只能**在 ON 中指定，**不能**在 WHERE 中
- 当属性名有歧义时，**必须**用表名限定（如 $Loan.loanNo$ ）

2.8.5 集合操作

```

-- 并（去重）
SELECT clientId FROM Depositor UNION SELECT clientId FROM Borrower;

-- 交（去重）
SELECT clientId FROM Depositor INTERSECT SELECT clientId FROM Borrower;

-- 差（去重）- Oracle 用 MINUS, SQL标准用 EXCEPT
SELECT clientId FROM Depositor MINUS SELECT clientId FROM Borrower;

```

- 默认**自动去重**；加 ALL 保留所有重复（如 UNION ALL）
- 要求**并兼容**（同属性数 + 同类型）

2.8.6 重命名

```

-- 属性重命名
SELECT clientId, loanNo AS loanId FROM Borrower;

```

```
-- 关系重命名（别名 / correlation name）
```

```
SELECT B.loanNo FROM Borrower B, Loan L WHERE B.loanNo = L.loanNo;
```

- Oracle 中别名的 `AS` 关键字在 `FROM` 子句中**不允许**
- **自连接**必须用别名来区分同一个表的不同实例

2.8.7 ORDER BY — 排序

```
SELECT name, rating FROM Client ORDER BY rating DESC, name ASC;
```

- `ASC` 默认升序，`DESC` 降序
- `null` 值默认是**最大值**（升序在最后，降序在最前）
- 可用 `NULLS FIRST` / `NULLS LAST` 调整

2.8.8 FETCH — 限制结果行数（Oracle）

```
-- 前3行
```

```
SELECT * FROM Client ORDER BY rating DESC FETCH FIRST 3 ROWS ONLY;
```

```
-- 前3行（含并列）
```

```
SELECT * FROM Client ORDER BY rating DESC FETCH FIRST 3 ROWS WITH TIES;
```

```
-- 跳过2行后取1行（第三高）
```

```
SELECT DISTINCT rating FROM Client ORDER BY rating DESC  
  OFFSET 2 ROWS FETCH NEXT 1 ROW ONLY;
```

通用语法: `[OFFSET n ROWS] FETCH [FIRST | NEXT] [n ROWS | PERCENT n] [ONLY | WITH TIES]`

2.8.9 CASE 语句

```
SELECT name,  
  CASE WHEN rating <= 3 THEN 'high risk'  
        WHEN rating <= 7 THEN 'medium risk'  
        WHEN rating <= 10 THEN 'low risk'  
        ELSE 'unknown risk'  
  END AS riskCategory  
FROM Client;
```

- 返回第一个匹配 `WHEN` 的值，无匹配时返回 `ELSE`（或 `null`）

2.9 Lecture 9: SQL DML — 聚合与子查询

2.9.1 聚合函数

函数	说明	注意事项
COUNT(*)	统计元组数	包含 null
COUNT(a)	统计非 null 值数	忽略 null
COUNT(DISTINCT a)	统计唯一非 null 值数	
SUM(a)	求和	忽略 null, 只能是数字
AVG(a)	平均值	忽略 null, 只能是数字
MAX(a) / MIN(a)	最大/最小值	忽略 null
MEDIAN(a)	中位数	Oracle
STDEV(a)	标准差	忽略 null

重要规则：

- 除 COUNT(*) 外，所有聚合函数忽略 null
- 空集合的聚合返回 null，除了 COUNT 返回 0
- COUNT(DISTINCT *) 是非法语法

2.9.2 GROUP BY — 分组聚合

```
SELECT branchName, COUNT(*), AVG(balance)
FROM Account
GROUP BY branchName;
```

关键规则：

- SELECT 中的非聚合属性必须出现在 GROUP BY 中
- GROUP BY 中的属性不必出现在 SELECT 中

概念执行顺序： *FROM* → *WHERE* → GROUP BY（形成分组）→ *HAVING* → *SELECT* → *ORDER BY*

2.9.3 HAVING — 分组过滤

```
SELECT branchName, AVG(balance) as avgBal
FROM Account
GROUP BY branchName
HAVING AVG(balance) > 55000;
```

- HAVING 只能与 GROUP BY 一起使用
- HAVING 中的条件在分组之后评估
- WHERE 在分组之前过滤元组；HAVING 在分组之后过滤分组

SQL 查询评估顺序：

1. FROM → 获取关系
2. WHERE → 过滤元组（分组前）
3. GROUP BY → 形成分组
4. HAVING → 过滤分组
5. SELECT → 计算聚合，选择输出
6. ORDER BY → 排序

注意：SELECT 中定义的别名不能被 WHERE/GROUP BY/HAVING 使用，因为它们评估更早。

检测重复：利用 GROUP BY + HAVING COUNT(*) 判断存在性/唯一性。

```
-- 只有一个账户的客户
SELECT clientId FROM Depositor d JOIN Account a ON d.accountNo=a.accountNo
WHERE branchName='Star House'
GROUP BY clientId HAVING COUNT(*) = 1;

-- 至少两个账户的客户
... HAVING COUNT(*) > 1;
```

2.9.4 嵌套子查询

核心思想：子查询返回关系，因此可以嵌套在任何需要值或集合的地方。

```
-- 比较运算符 + 标量子查询（必须返回单值）
SELECT * FROM Loan WHERE amount > (SELECT AVG(amount) FROM Loan);

-- 获取第三高评级的所有客户
SELECT name FROM Client
WHERE rating = (SELECT DISTINCT rating FROM Client
```

```
ORDER BY rating DESC NULLS LAST
OFFSET 2 ROWS FETCH NEXT 1 ROW ONLY);
```

2.9.5 集合成员测试

```
-- IN: 是否在集合中
SELECT clientId FROM Borrower
WHERE clientId IN (SELECT clientId FROM Depositor);

-- NOT IN: 是否不在集合中
SELECT clientId FROM Borrower
WHERE clientId NOT IN (SELECT clientId FROM Depositor);
```

集合对集合比较:

```
-- 找到每个分行的最高余额客户
SELECT branchName, clientId, name, accountNo, balance
FROM Client NATURAL JOIN Depositor NATURAL JOIN Account
WHERE (branchName, balance) IN
    (SELECT branchName, MAX(balance) FROM Account GROUP BY branchName);
```

2.9.6 SOME / ALL

操作	含义	等价
> SOME(set)	大于集合中至少一个	> MIN(set)
< SOME(set)	小于集合中至少一个	< MAX(set)
= SOME(set)	等于集合中至少一个	等价于 IN
<> SOME(set)	不等于集合中至少一个	不等价于 NOT IN
> ALL(set)	大于集合中所有值	> MAX(set)
< ALL(set)	小于集合中所有值	< MIN(set)
= ALL(set)	等于集合中所有值	不等价于 IN
<> ALL(set)	不等于集合中所有值	等价于 NOT IN

```
-- 资产大于 Yau Tsim Mong 所有分行的分行
SELECT branchName FROM Branch
WHERE assets > ALL (SELECT assets FROM Branch WHERE district='Yau Tsim
Mong');
```


2.9.7 EXISTS / NOT EXISTS

-- 同时有贷款和账户的客户（相关子查询 / correlated subquery）

```
SELECT clientId FROM Depositor D
WHERE EXISTS (SELECT * FROM Borrower B WHERE D.clientId = B.clientId);
```

-- 有账户但没有贷款的客户

```
SELECT clientId FROM Depositor D
WHERE NOT EXISTS (SELECT * FROM Borrower B WHERE D.clientId = B.clientId);
```

- NOT EXISTS 可以模拟**集合包含**：A 包含 B $\Leftrightarrow$ NOT EXISTS (B MINUS A)

作用域规则：子查询中可引用外层别名（如 D.clientId）；内层别名不能在外层使用。

2.9.8 FROM 子句中的子查询

```
SELECT branchName, avgBalance
FROM (SELECT branchName, AVG(balance) AS avgBalance
      FROM Account GROUP BY branchName) result
WHERE avgBalance > (SELECT AVG(balance) FROM Account);
```

- 结果被称为**派生关系**（derived/temporary relation），查询结束后丢弃
- Oracle 的作用域限制可能需要使用 WITH 子句代替

2.9.9 WITH 子句（CTE）

```
WITH result (branchName, avgBalance) AS
  (SELECT branchName, AVG(balance) FROM Account GROUP BY branchName)
SELECT branchName, avgBalance
FROM result
WHERE avgBalance = (SELECT MAX(avgBalance) FROM result);
```

- 定义**只在当前查询中可见**的临时关系
- 比 FROM 子查询更清晰，解决了 Oracle 的作用域限制

2.10 Lecture 10: SQL DML — 分析函数与数据修改

2.10.1 聚合函数 vs 分析函数

特性	聚合函数 (Aggregate)	分析函数 (Analytic)
返回行数	每组一行	每个输入行一行
需要 GROUP BY	是 (多行时)	不需要
可以分组	GROUP BY	PARTITION BY
出现位置	SELECT, HAVING	仅 SELECT 和 ORDER BY
执行顺序	—	最后执行 (ORDER BY 之前)

```

-- 聚合：每组一行（非法 - 没有 GROUP BY 但有非聚合属性）
SELECT accountNo, balance, SUM(balance) FROM Account; -- ✗

-- 分析：每个 account 行都显示合计值
SELECT accountNo, balance, SUM(balance) OVER () AS totalBalance FROM
Account; -- ✓

```

2.10.2 分析函数语法

```

analytic_function([args]) OVER (
  [PARTITION BY clause]
  [ORDER BY clause]
  [windowing_clause]
)

```

子句	作用
PARTITION BY	将查询结果分区 (类似 GROUP BY, 但不合并行)
ORDER BY	指定分区内的排序
windowing_clause	指定滑动窗口

2.10.3 RANK / DENSE_RANK

函数	并列处理	是否有"空缺"
RANK()	相同值同排名	有 (1, 2, 2, 4, ...)
DENSE_RANK()	相同值同排名	无 (1, 2, 2, 3, ...)
ROW_NUMBER()	并列随机分配	无, 始终唯一
PERCENT_RANK()	$(rank-1)/(n-1)$ 分数形式	
CUME_DIST()	p/n 累积分布	

Top-N 查询：将排序查询嵌套后过滤排名。

```
SELECT branchName, assets FROM
  (SELECT branchName, assets, RANK() OVER (ORDER BY assets DESC) r FROM
    Branch)
WHERE r <= 3;
```

2.10.4 NTILE

将元组**均匀分配**到 n 个桶中（用于百分位数分析）。

```
SELECT branchName, assets,
  NTILE(4) OVER (ORDER BY assets DESC) AS quartile
FROM Branch;
-- 如果总数不能被 n 整除，各桶最多差 1
```

2.10.5 LISTAGG

将多行数据连接为一个分隔列表。

```
-- 聚合模式：每个 district 一行，客户名列表
SELECT district, LISTAGG(name, ', ') WITHIN GROUP (ORDER BY name) AS clients
FROM Client GROUP BY district;

-- 分析模式：每个 loan 行都显示同一分行的所有金额列表
SELECT loanNo, amount, branchName,
  LISTAGG(amount, ', ') WITHIN GROUP (ORDER BY amount)
  OVER (PARTITION BY branchName) AS "all loan amounts"
FROM Loan WHERE year='2025';
```

2.10.6 窗口 (Windowing)

概念：在分析函数的分区内定义一个**滑动窗口**，每个元组基于其窗口内的数据计算函数值。窗口随当前元组移动。

```
SELECT year,
  AVG(yearLoanTotal) OVER (
    ORDER BY year
    ROWS BETWEEN UNBOUNDED PRECEDING AND 1 FOLLOWING
  ) AS movingAvg
FROM (SELECT year, SUM(amount) AS yearLoanTotal FROM Loan GROUP BY year);
```

窗口语法： ROWS | RANGE BETWEEN <start> AND <end>

边界	含义
UNBOUNDED PRECEDING	分区第一行
UNBOUNDED FOLLOWING	分区最后一行
CURRENT ROW	当前行/值
n PRECEDING	前 n 行 (ROWS) 或前 n 范围 (RANGE)
n FOLLOWING	后 n 行 (ROWS) 或后 n 范围 (RANGE)

区别	ROWS	RANGE
单位	物理行 (行数)	逻辑值 (值偏移)
相同值处理	每行独立窗口	相同排序值的行共享窗口

2.10.7 数据修改

INSERT — 插入元组

```
-- 单行插入（按属性顺序）
INSERT INTO Account VALUES ('A-332', 1200, 'Pacific Place');

-- 指定属性插入（推荐，顺序无关）
INSERT INTO Account (accountNo, branchName, balance) VALUES ('A-334',
'Pacific Place', 1200);

-- 从查询结果批量插入
INSERT INTO Account
  SELECT loanNo, 200, branchName FROM Loan WHERE branchName='Pacific Place';
```

注意：从查询插入时**不能用** VALUES 关键字。

DELETE — 删除元组

```
-- 条件删除
DELETE FROM Account WHERE branchName='Pacific Place';

-- 全表删除
DELETE FROM Account; -- 删除所有元组！
```

```
-- 复杂删除（引用子查询）
DELETE FROM Depositor
WHERE accountNo IN (SELECT accountNo FROM Depositor NATURAL JOIN Account
                    WHERE branchName='Star House');
```

UPDATE — 更新元组

```
-- 条件更新
UPDATE Account SET balance = 50000 WHERE accountNo = 'A-333';

-- 批量更新
UPDATE Account SET balance = balance * 1.05 WHERE balance < 10000;

-- CASE 合并多次更新
UPDATE Account SET balance = CASE
    WHEN balance <= 10000 THEN balance * 1.05
    ELSE balance * 1.06
END;
```

用 `CASE` 替代多条 UPDATE 语句，避免**执行顺序**问题。

2.11 Lecture 11: SQL DML & DDL — 过程化 SQL 与模式定义

2.11.1 DBMS API 与 PL/SQL

DBMS API：客户端应用通过网络与 DBMS 通信（如 Oracle 使用 port 1521）。

Oracle PL/SQL：类似 C 的过程式编程语言，SQL 语句嵌入式使用。

- **过程 (Procedure)**：不返回值，用 `exec` 调用
- **函数 (Function)**：用 `RETURN` 返回值

PL/SQL 基本结构

```
CREATE OR REPLACE PROCEDURE proc_name [ AS | IS ]
-- 声明段（变量、类型、游标）
BEGIN
-- 执行段（必须存在）
-- 允许：SELECT, INSERT, UPDATE, DELETE
-- 不允许：CREATE, DROP, ALTER, RENAME
EXCEPTION
```

```
-- 异常处理段
END;
```

内容摘要

- **数据类型**：NUMBER, INT, CHAR, VARCHAR2 等；也可用 table.column%TYPE 或 table%ROWTYPE
- **控制流**：IF-THEN-ELSIF, CASE, LOOP, WHILE, FOR, EXIT, CONTINUE, GOTO
- **SELECT INTO**：将查询结果赋值给程序变量，**必须只返回一行**，否则触发异常

2.11.2 游标 (Cursor)

游标用于在 PL/SQL 中**逐行**处理多行查询结果。

```
-- 声明
CURSOR cursor_name IS select_statement;

-- 隐式迭代（推荐）
FOR record IN cursor_name LOOP
    -- record.column_name 访问当前行的值
END LOOP;

-- 显式管理
OPEN cursor_name;
FETCH cursor_name INTO variables;
CLOSE cursor_name;
```

游标状态：%FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT

2.11.3 异常处理

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN ... -- SELECT INTO 返回零行
    WHEN TOO_MANY_ROWS THEN ... -- SELECT INTO 返回多行
    WHEN DUP_VAL_ON_INDEX THEN ... -- 主键重复
    WHEN OTHERS THEN ... -- 任何其他异常
```

可自定义异常：exception_name EXCEPTION; → RAISE exception_name;

2.11.4 SQL DDL — 数据定义语言

语句	用途
CREATE TABLE	创建关系模式
ALTER TABLE ... ADD	添加属性
ALTER TABLE ... DROP COLUMN	删除属性
DROP TABLE	删除关系（模式+数据）

基本域类型

类型	说明
CHAR(n)	定长字符串（空格填充）
VARCHAR2(n)	变长字符串（Oracle 推荐；标准用 VARCHAR）
INT	整数
NUMBER(p, d)	定点数（p=总位数, d=小数位）
FLOAT(n)	浮点数
DATE	日期（Oracle 包含时间）
TIMESTAMP	日期+时间

2.11.5 完整性约束 (Integrity Constraints)

约束	关键字	说明
NOT NULL	NOT NULL	属性不能为 null
PRIMARY KEY	PRIMARY KEY	主键：唯一 + 非空
UNIQUE	UNIQUE	候选键：唯一（可为 null）
FOREIGN KEY	REFERENCES T(k)	外键：值必须匹配被引用表的主键或为 null
CHECK	CHECK (P)	自定义谓词条件

2.11.6 外键操作 (Referential Actions)

```

FOREIGN KEY (accountNo) REFERENCES Account(accountNo)
ON DELETE CASCADE      -- 删除被引用元组时，级联删除引用元组
-- 或 ON DELETE SET NULL -- 删除被引用元组时，将外键设为 null
-- 或 ON DELETE SET DEFAULT -- 删除被引用元组时，将外键设为默认值
-- 或无（默认）         -- 禁止删除被引用的元组

```

完整选项：

- `ON DELETE` : CASCADE / SET NULL / SET DEFAULT / (默认拒绝)
- `ON UPDATE` : CASCADE (**Oracle 不支持**) / (默认拒绝)

限制：如果外键是主键的一部分，不能使用 SET NULL 或 SET DEFAULT。

2.11.7 CHECK 约束

```
CREATE TABLE Loan (  
  loanNo CHAR(5) PRIMARY KEY,  
  amount NUMBER(8,2) CHECK (amount >= 1000 AND amount <= 100000),  
  year CHAR(4),  
  branchName VARCHAR2(15) NOT NULL  
);
```

2.11.8 视图 (Views)

视图 = 通过查询定义的"虚拟关系"，用于**隐藏数据**。

```
-- 创建视图  
CREATE VIEW BranchLoan AS  
  SELECT loanNo, year, branchName FROM Loan; -- 隐藏 amount  
  
-- 查询视图（如普通表）  
SELECT loanNo FROM BranchLoan WHERE branchName='Star House';  
  
-- 删除视图  
DROP VIEW BranchLoan;
```

⚠ **重要：**视图**绝不应该**用来表达查询！视图是用来控制数据访问的。

可更新视图的条件（所有条件都必须满足）：

1. `FROM` 子句只包含**一个**关系
2. `SELECT` 只包含属性名（无表达式、聚合、DISTINCT）
3. 未列出的属性可以设为 null
4. 查询没有 `GROUP BY` 或 `HAVING`

2.11.9 断言 (Assertions)


```
CREATE ASSERTION loanSumConstraint AS CHECK
(NOT EXISTS
  (SELECT * FROM Branch
    WHERE (SELECT SUM(amount) FROM Loan NATURAL JOIN Branch)
      >=
      (SELECT SUM(balance) FROM Account NATURAL JOIN Branch))));
```

- 断言可能涉及**多个关系**，在**任何更新**时检查
- 开销很大，应谨慎使用
-  **Oracle 不支持断言**

2.11.10 触发器 (Triggers)

触发器由数据库修改事件**自动执行**，用于实现其他约束无法表达的完整性规则。

```
CREATE OR REPLACE TRIGGER overdraft
  BEFORE UPDATE OF balance ON Account
  FOR EACH ROW
  WHEN (NEW.balance < 0)
DECLARE
  currentYear Loan.year%TYPE;
BEGIN
  SELECT TO_CHAR(SYSDATE, 'YYYY') INTO currentYear FROM DUAL;
  INSERT INTO Loan VALUES (:OLD.accountNo, -:NEW.balance, currentYear,
:OLD.branchName);
  INSERT INTO Borrower (SELECT clientName, accountNo FROM Depositor
                        WHERE accountNo = :OLD.accountNo);
  :NEW.balance := 0;
END;
```

触发器语法要素：

要素	说明
‘BEFORE	AFTER’
‘DELETE	INSERT
FOR EACH ROW	逐行触发（省略则语句级触发一次）
WHEN (condition)	触发条件
:OLD.col / :NEW.col	修改前/后的值（在 PL/SQL 中需加 :）

用途：实现跨表约束、审计日志、自动维护聚合值（如分行负债总额）、业务规则自动化。

2.11.11 SQL 核心知识地图

SQL 查询基础 (L8)

- └─ SELECT-FROM-WHERE 结构
- └─ 投影：DISTINCT, 算术运算, COALESCE
- └─ 选择：比较, BETWEEN, LIKE, REGEXP_LIKE, NULL
- └─ 连接：NATURAL JOIN, JOIN ON/USING, OUTER JOIN
- └─ 集合：UNION, INTERSECT, EXCEPT/MINUS
- └─ 重命名：AS (列), 别名 (表)
- └─ 排序限制：ORDER BY, FETCH/OFFSET
- └─ 条件逻辑：CASE

聚合与子查询 (L9)

- └─ 聚合函数：COUNT, SUM, AVG, MAX, MIN
- └─ GROUP BY + HAVING
- └─ 嵌套子查询
- └─ 集合成员：IN, NOT IN
- └─ 集合比较：SOME, ALL
- └─ 存在性：EXISTS, NOT EXISTS
- └─ FROM 子查询 (派生表)
- └─ WITH 子句 (CTE)

分析函数与数据修改 (L10)

- └─ 分析函数 vs 聚合函数
- └─ RANK, DENSE_RANK, ROW_NUMBER
- └─ NTILE (分桶)
- └─ LISTAGG (列表聚合)
- └─ 窗口：ROWS/RANGE BETWEEN
- └─ INSERT / DELETE / UPDATE
- └─ CASE 合并更新

DDL 与过程化 SQL (L11)

- └─ PL/SQL：过程, 函数, 块结构
- └─ 游标：显式/隐式, FOR LOOP
- └─ 异常处理
- └─ DDL：CREATE/ALTER/DROP TABLE
- └─ 域类型：CHAR, VARCHAR2, NUMBER, DATE, TIMESTAMP
- └─ 约束：PK, FK, UNIQUE, CHECK, NOT NULL
- └─ 外键操作：CASCADE, SET NULL, SET DEFAULT
- └─ 视图：创建/查询/更新条件
- └─ 断言 (Oracle 不支持)
- └─ 触发器：BEFORE/AFTER, FOR EACH ROW, :OLD/:NEW

时间: 2026年7月3日–11日

覆盖: Lecture 12 – Lecture 15 (NoSQL 数据库系统 & MongoDB)

MongoDB 示例数据库 (Bank Schema) :

```
clients: {clientId, name, hkid, address, district, rating,
  accounts: [{accountNo, balance, branch}],
  loans: [{loanNo, amount, year, branch}],
  tags: [...]}
branches: {branch, district, liabilities, assets,
  accounts: [...], loans: [...]}
```

3.12 Lecture 12: NoSQL 数据库管理系统

3.12.1 NoSQL 的动机：大数据 (Big Data)

大数据的三个维度 (3V)：

维度	含义
Volume (容量)	数据量极大，持续增长
Velocity (速度)	数据产生和数据分析速度极快
Variety (多样性)	非结构化、半结构化数据，超越关系型

- 许多应用**愿意牺牲关系型 DBMS 特性**以获得高可扩展性

3.12.2 NoSQL 运动核心思想

RDBMS (关系型)	NoSQL
强调数据 一致性	接受 最终一致性
正式模式、类型、参照完整性、事务	灵活模式或无模式
垂直扩展 为主 (增加单机能力)	水平扩展 (增加节点数)
水平扩展时协调开销大	近线性的水平扩展 + 高可用性
丰富查询功能 (SQL)	简单查询 / API

最终一致性 (Eventual Consistency)：数据及其副本在事务后的某个时间点最终达到一致，不保证瞬时一致性。

3.12.3 四种 NoSQL 类型

类型	数据组织	代表系统
Key-Value	唯一键 → 无模式值（字节串）	BerkeleyDB, Redis, Memcached, Riak
Document	键 → 半结构化文档（JSON/BSON/XML）	MongoDB, CouchDB, OrientDB
Columnar	按列组织（非按行），null 不占空间	Cassandra, HBase, Amazon SimpleDB
Graph	节点 + 边，支持图遍历查询	Neo4J, FlockDB, OrientDB

Key-Value DBMS

- 基本操作： `put(key, value)`, `get(key)`, `delete(key)`
- 键是**唯一搜索标准**
- 值通常为无模式字节，DBMS 不解释其内容

Document DBMS

- 一种 Key-Value DBMS，值是 JSON-like 键值对集合
- 支持**非键属性查询**
- JSON 最广泛使用

Columnar DBMS

- 按**列**而非**行**组织数据
- null 不占存储空间
- 可直接按列值筛选（无需索引）

Graph DBMS

- N:M 关系自然映射为图结构，不需要连接表
- 查询使用图模式匹配（如 Cypher 语言）

3.12.4 JSON 语法

元素	语法	说明
对象 Object	{ }	无序键值对集合，键必须用双引号
数组 Array	[]	无序值集合，逗号分隔
字符串 String	" "	双引号，反斜杠转义
数字 Number	—	整数、实数、科学记数法（不能有多余零）
布尔 Boolean	true / false	—
Null	null	—

3.12.5 Oracle SQL/JSON 函数

函数	功能	默认 null 处理
json_object	从查询元组构造 JSON 对象集合 （每个元组一个对象）	null on null
json_array	从查询元组构造 JSON 数组集合 （每个元组一个数组）	absent on null
json_objectagg	聚合多行为 单个 JSON 对象	null on null
json_arrayagg	聚合多行为 单个 JSON 数组	absent on null

可选子句：

- null on null / absent on null — 控制 SQL null 的处理
- returning clause — 指定返回类型（默认 varchar2(4000)）
- strict — 检查输出是否为合法 JSON
- with unique keys — 确保 JSON 键无重复

3.12.6 RDBMS vs NoSQL 总结

特性	RDBMS	NoSQL
数据组织	关系表	Key-value / Document / Column / Graph
分布	通常单服务器	多分布式服务器
扩展性	垂直扩展，难以水平扩展	易于水平扩展，易于复制
模式	模式驱动	无模式 / 灵活模式
查询语言	SQL	简单 API 或专用语言
功能集	丰富（触发器、视图、存储过程等）	简单 API
数据量	正常规模	海量数据 / 极高读写频率

3.13 Lecture 13: MongoDB — 数据模型与查询语言

3.13.1 MongoDB 数据模型

- 数据库 (Database)
 - 集合 (Collection) - 类似于关系表
 - 文档 (Document) - 类似于行, JSON-like (BSON)
 - 字段 (Field) - 键值对

核心概念：

概念	关系型	MongoDB
表	Table	Collection
行	Row	Document
列	Column	Field
主键	Primary Key	_id (自动生成或用户指定)
模式	Schema-driven	Schemaless (灵活模式)

MongoDB 额外数据类型：Date, BinData, Regular Expression, Object ID

3.13.2 MongoDB 数据库设计

关系表示方式

方式	说明	优点	缺点
References (引用)	文档中包含其他文档的 ID 链接	数据规范化, 适合 N:M 关系	需要多次查询
Embedded Data (嵌入式)	相关数据存储在同一文档中	单次操作即可读写, 读性能好	数据重复 (反规范化)

选择依据：取决于应用的最常见查询模式。

设计关键：集合中文档的字段可以不同，字段的数据类型在文档间也可以不同（但有模式验证可强制一致性）。

3.13.3 查询文档

```
db.collection.find(queryFilter, projection)
db.collection.findOne(queryFilter, projection)
```

- `find` 返回所有匹配文档； `findOne` 返回一个匹配文档
- `queryFilter` 为空则返回所有文档

3.13.4 投影 (Projection)

规则	说明
值 = 1 (或 true)	包含 该字段
值 = 0 (或 false)	排除 该字段
默认	<code>_id</code> 始终包含 (除非显式排除)
互斥规则	除 <code>_id</code> 外, 包含和排除 不能同时指定

3.13.5 比较表达式操作符

操作符	含义
<code>\$eq</code>	等于 (默认, 可省略)
<code>\$ne</code>	不等于
<code>\$lt</code> / <code>\$lte</code>	小于 / 小于等于
<code>\$gt</code> / <code>\$gte</code>	大于 / 大于等于
<code>\$cmp</code>	比较两个值 (返回 -1, 0, 1)

3.13.6 布尔表达式操作符

操作符	语法	说明
<code>\$and</code>	<code>{ \$and: [{p1}, {p2}, ...] }</code>	逻辑与 (默认行为)
<code>\$or</code>	<code>{ \$or: [{p1}, {p2}, ...] }</code>	逻辑或
<code>\$nor</code>	<code>{ \$nor: [{p1}, {p2}, ...] }</code>	逻辑非或
<code>\$not</code>	<code>{ \$not: {predicate} }</code>	逻辑非

重要陷阱：

- 默认 AND 对**同名字段**的多个条件：第二个条件**覆盖**第一个条件
 - ❌ `{rating: {$gte: 7}, rating: {$lte: 9}}` → 只应用了 `$lte: 9`
 - ✅ `{$and: [{rating: {$gte: 7}}, {rating: {$lte: 9}}]}`
 - ✅ `{rating: {$gte: 7, $lte: 9}}` — 同名字段可用数组写法

3.13.7 条件表达式操作符

操作符	用途
<code>\$ifNull</code>	返回第一个非 null 表达式值 / 替代值
<code>\$cond</code>	if-then-else; 返回 trueCase / falseCase
<code>\$switch</code>	多分支条件判断 (branches 数组)

`$cond` 语法：

```
{$cond: {if: booleanExpr, then: trueCase, else: falseCase}}
```

// 或数组语法

```
{$cond: [booleanExpr, trueCase, falseCase]}
```

3.13.8 查询谓词操作符 `$expr`

- 允许在查询条件中使用**聚合表达式**
- 用于**比较同一文档的两个字段**或字段计算后比较

```
// 资产 < 负债的分行
db.branches.find({$expr: {$lt: ['$assets', '$liabilities']}})
```

3.13.9 正则表达式 `$regex`

```
// 名字以 Steven 或 Stephen 开头
db.clients.find({name: {$regex: /^Ste(v|ph)en/}})
```

- 选项：'i' 不区分大小写

3.13.10 Null/缺失 比较

查询	匹配条件
<code>{field: null}</code>	null 或 字段不存在
<code>{field: {\$ne: null}}</code>	字段存在且非 null
<code>{field: {\$exists: true}}</code>	字段存在（含 null）
<code>{field: {\$exists: false}}</code>	字段不存在

3.14 Lecture 14: MongoDB — 数组查询与模式验证

3.14.1 查询数组元素

访问方式	语法	说明
按位置	<code>'loans.0.amount'</code>	访问数组第 1 个元素的 amount 字段
按字段名	<code>'loans.amount'</code>	数组中至少一个元素满足条件

关键警告：

- 对数组字段的 AND 条件可由不同数组元素满足（不是同一元素的 AND）
- 解决方案：使用 `$elemMatch`

3.14.2 数组查询操作符

操作符	功能
<code>\$elemMatch</code>	至少一个数组元素同时满足所有条件
<code>\$all</code>	数组包含所有指定值（任意顺序）
<code>\$size</code>	数组长度等于指定数字
<code>\$in / \$nin</code>	字段值在 / 不在指定数组中
<code>\$slice</code>	返回数组的前 / 后 N 个元素或跳过 N 个后返回 M 个

`$elemMatch` 在投影中：只返回第一个匹配条件的数组元素。

3.14.3 数组表达式操作符

操作符	功能
<code>\$size</code>	返回数组元素个数
<code>\$isArray</code>	判断是否为数组
<code>\$arrayElemAt</code>	返回指定索引位置的元素（支持负索引）
<code>\$first / \$last</code>	返回数组第一个 / 最后一个元素
<code>\$firstN / \$lastN</code>	返回数组前 / 后 N 个元素
<code>\$minN / \$maxN</code>	返回数组最小 / 最大的 N 个值
<code>\$filter</code>	筛选数组中满足条件的元素
<code>\$map</code>	对数组每个元素应用表达式
<code>\$reduce</code>	将数组元素合并为单个值
<code>\$concatArrays</code>	拼接多个数组
<code>\$sortBy</code>	对数组元素排序

3.14.4 集合表达式操作符

操作符	功能
<code>\$setDifference</code>	只在 array1 中的唯一元素
<code>\$setIntersection</code>	所有数组共有唯一元素
<code>\$setUnion</code>	所有数组中的唯一元素
<code>\$setEquals</code>	是否有相同不同元素
<code>\$setIsSubset</code>	array1 是否为 array2 子集
<code>\$allElementsTrue</code>	是否无元素为 false
<code>\$anyElementTrue</code>	是否有元素为 true

3.14.5 去重、计数、排序、限制

方法	功能
<code>distinct(field)</code>	返回指定字段的唯一值（作为数组）
<code>count()</code>	返回查询结果文档数量
<code>sort({field: 1/-1})</code>	按字段排序（1=升序, -1=降序）
<code>limit(n)</code>	限制结果为 n 个文档
<code>skip(n)</code>	跳过前 n 个文档

Sort 注意事项:

- 排序只有在**存在**于输入文档中的字段上进行（不能在投影的计算字段上排序）
- sort 总是在 projection 之前执行
- null/缺失字段值视为**最小值**

3.14.6 JSON Schema 验证

MongoDB 使用 `$jsonSchema` 定义文档验证规则:

关键字	含义
<code>bsonType</code>	BSON 数据类型（如 <code>'int'</code> , <code>'string'</code> , <code>'array'</code> , <code>'object'</code> ）
<code>required</code>	必须存在的字段数组
<code>properties</code>	每个字段的模式定义
<code>enum</code>	字符串值枚举列表
<code>minimum</code> / <code>maximum</code>	数值范围
<code>minLength</code> / <code>maxLength</code>	字符串/数组长度限制
<code>pattern</code>	正则表达式匹配
<code>description</code>	验证失败时的错误提示
<code>uniqueItems</code>	数组元素必须唯一

3.15 Lecture 15: MongoDB — 聚合框架

3.15.1 聚合框架概念

聚合框架 = **数据处理流水线**: 文档流经多个阶段，每个阶段执行一个操作。

```
input documents → stage 1 → stage 2 → ... → stage n → output documents
```

- 每个阶段的输入和输出都是**文档**
- 聚合方法: `db.collection.aggregate([pipeline])`

3.15.2 常用聚合阶段

阶段	功能	SQL 对应
\$project	限制/变换字段	SELECT
\$match	过滤文档	WHERE/HAVING
\$group	按标识符分组，应用累加器	GROUP BY
\$count	计算文档数	COUNT
\$sort	排序	ORDER BY
\$limit	限制输出文档数	FETCH
\$skip	跳过前 N 个文档	OFFSET
\$unwind	数组展开（一个元素 → 一个文档）	N/A
\$lookup	左外连接	LEFT JOIN

3.15.3 累加器 (Accumulators)

累加器	功能
\$count	计数
\$sum	求和
\$avg	平均值
\$max / \$min	最大 / 最小值
\$median	近似中位数
\$stdDevPop	总体标准差
\$addToSet	添加值到数组（无重复）
\$push	追加值到数组（允许重复）

- 在 \$project 中：对单个文档的数组字段计算
- 在 \$group 中：对多个文档跨文档计算

3.15.4 \$project 阶段

```
{ $project: { _id: 0, district: 1 } }
```

- 输出仅含指定字段（与 find 投影相同）
- 但不能是空文档

3.15.5 \$match 阶段

```
{ $match: { district: 'Islands' } }
```

- 与 find 的 queryFilter 相同
- **一般不允许聚合表达式** (`$expr` 例外)

3.15.6 `$sort`、`$limit`、`$skip` 阶段

与同名方法功能相同，但：

- **不能使用聚合表达式**
- 与 `find` 中的方法不同：执行顺序**由阶段顺序决定**（而非强制 `sort` → `skip` → `limit`）

3.15.7 `$group` 阶段

```
{ $group: {  
  _id: groupKey,           // 分组键; null → 全局聚合  
  field: { accumulator: expr }  
} }
```

- 每个唯一分组键输出一个文档
- 复合键需用 `{ field1: '$field1', field2: '$field2' }` 格式

3.15.8 `$count` 阶段

```
{ $count: 'fieldName' }
```

- 计算到达该阶段的文档数
- 输出一个文档，其中 `fieldName` 为计数值

3.15.9 `$unwind` 阶段 —— 关键！

将数组字段**展开**为多个文档（每个数组元素一个文档）。

```
{ $unwind: { path: '$accounts' } }  
// 或保留 null/空数组的文档：  
{ $unwind: { path: '$accounts', preserveNullAndEmptyArrays: true } }
```

为什么需要 unwind？ — 累加器无法直接应用于数组字段。

\$unwind vs \$match 顺序的重要性:

- `$unwind` → `$match`: 在每个展开的数组元素上过滤 → **正确**
- `$match` → `$unwind`: 匹配整个文档（数组包含至少一个匹配元素） → 可能包含不想要的元素

3.15.10 \$addToSet — 去重

```
{ $group: {  
  _id: null,  
  uniqueAccounts: { $addToSet: { accountNo: '$accounts.accountNo', ... } }  
}}
```

- 保证数组中没有重复元素
- 常用于消除重复（如同一账户被多个客户持有）

3.15.11 \$push — 保留字段

```
{ $group: {  
  _id: '$rating',  
  clients: { $push: { clientId: '$clientId', name: '$name' } }  
}}
```

- 在分组时保留输入文档的字段信息
- `$each` 修饰符可一次添加多个值
- `$slice`、`$sort`、`$position` 修饰符可用（需配合 `$each`）

3.15.12 \$lookup — 左外连接

```
{ $lookup: {  
  from: 'branches', // 被连接的集合  
  localField: 'accounts.branch', // 输入文档的字段  
  foreignField: 'branch', // 被连接集合的字段  
  as: 'branchInfo' // 输出数组字段名  
}}
```

- 为每个输入文档**添加一个数组字段**，包含匹配的文档
- 相当于 SQL 的 LEFT OUTER JOIN

3.15.13 聚合框架流水线设计模式

计算去重的跨分支总额：

1. `$unwind` 展开数组
2. `$group` + `$addToSet` 去重
3. `$unwind` 再次展开去重后的数组
4. `$group` + `$sum` 聚合

找到每组的最大值获得者：

1. `$unwind` → `$group` (+ `$push` 保留信息 + `$max` 记录最大值)
2. `$unwind` → `$match` (用 `$expr` 比较)

模拟 Top-N (含并列)：

1. `$group` + `$push` → `$sort` → `$skip` → `$limit`

3.15.14 NoSQL & MongoDB 核心知识地图

NoSQL 基础 (L12)

- └─ 大数据 3V: Volume, Velocity, Variety
- └─ NoSQL 核心理念: 最终一致性、水平扩展、灵活模式
- └─ 四种类型: Key-Value, Document, Columnar, Graph
- └─ JSON 语法: 对象 {}, 数组 [], 六种数据类型
- └─ Oracle SQL/JSON 函数: json_object, json_array, json_objectagg, json_arrayagg

MongoDB 数据模型 (L13)

- └─ 数据库 → 集合 → 文档 → 字段
- └─ `_id` (自动或用户定义, 唯一)
- └─ References vs Embedded Data
- └─ 聚合表达式: `$` 字段路径, `$` 操作符, `$$` 变量
- └─ 基本查询: `find/findOne` + `queryFilter` + `projection`

MongoDB 查询语言 (L13-L14)

- └─ 比较: `$eq`, `$ne`, `$lt`, `$lte`, `$gt`, `$gte`, `$cmp`
- └─ 布尔: `$and`, `$or`, `$nor`, `$not`
- └─ 条件: `$ifNull`, `$cond`, `$switch`
- └─ 谓词: `$expr` (跨字段比较)
- └─ 正则: `$regex`
- └─ Null/缺失: `null`, `$exists`, `$ne: null`
- └─ 数组查询: 位置访问, `$elemMatch`, `$all`, `$size`
- └─ 数组表达式: `$filter`, `$map`, `$reduce`, `$sortBy`
- └─ 集合操作: `$setDifference`, `$setIntersection`, `$setUnion`
- └─ 结果处理: `distinct`, `count`, `sort`, `limit`, `skip`

└─ 模式验证: `$jsonSchema + bsonType`

MongoDB 聚合框架 (L15)

- └─ 流水线概念: 阶段序列处理文档
- └─ `$project`: 限制/变换字段
- └─ `$match`: 过滤文档
- └─ `$group`: 分组聚合 (`_id` + 累加器)
- └─ `$count`: 计数
- └─ `$sort` / `$limit` / `$skip`: 排序和限制
- └─ `$unwind`: 数组展开 (核心!)
- └─ `$addToSet`: 去重收集
- └─ `$push`: 保留字段信息
- └─ `$lookup`: 左外连接
- └─ 累加器: `$sum`, `$avg`, `$max`, `$min`, `$median`, `$addToSet`, `$push`

Part 4: Lectures 16–24 — 存储、索引、查询处理、事务与恢复

时间: 2026年7月12日–24日

覆

盖: Lecture 16 – Lecture 24 (存储与文件结构、索引、查询处理、事务管理与恢复系统)

参考符号约定:

B = 文件页数	nr = 元组数	M = 缓冲区页数
bfr = 每页元组数	$V(A, r) = A$ 在 r 中不同值数	
$HT_i = B^+tree$ 索引高度	fi = 内部节点平均扇出	

4.16 Lecture 16: 存储与文件结构

4.16.1 存储设备层次

访问速度 快 / 成本 高

- └─ Cache (缓存)
- └─ Main Memory / RAM (主存) ← 主存储 (Primary), 易失
- └─ SSD (固态硬盘) ⇕ I/O 边界
- └─ HDD (机械硬盘) ← 辅存储 (Secondary), 持久
- └─ 光盘 / 磁带 ← 第三存储 (Tertiary)

访问速度 慢 / 成本 低

- **辅存储**以**页/块 (page/block)**为单位读写 (通常 512B–16KB)
- 辅存储 I/O 远慢于主存操作 → DBMS 性能瓶颈

4.16.2 数据库缓冲区 (Database Buffer)

概念	说明
缓冲区 (Buffer)	主存中用于存储辅存储页面副本的区域
缓冲区管理器 (Buffer Manager)	操作系统子系统，管理缓冲空间
目标	最小化辅存储访问次数

请求页面的流程：

- 1. 若页已在缓冲区 → 直接返回地址
- 2. 若不在 → 分配空间（可能需要替换页面）→ 从辅存储读入

页面替换策略：

- **LRU** (Least Recently Used)：替换最久未使用的页
- **MRU** (Most Recently Used)：对嵌套循环等模式更好（如计算 join 时反复扫描同一关系）

4.16.3 记录组织

定长记录 (Fixed-Length Records)

组织方式	特点
相对位置	记录 i 从字节 $n \times (i-1)$ 开始；删除时移动记录（破坏指针）
空闲链表 (Free List)	删除时不移动记录，将第一个删除记录地址存在文件头中（空间高效）

- 文件阻塞因子： $bfr = \lfloor \text{页大小} / \text{记录大小} \rfloor$
- 所需页数： $\lceil \text{\#记录} / bfr \rceil$

变长记录 (Variable-Length Records)

方式	特点
字节串表示	顺序存储，附加结束标记；删除导致碎片
嵌入式标识	属性名前缀每个字段；额外空间但高效处理缺失字段
指针方法	锚页 + 溢出页，指针将相关记录链接

方式	特点
槽页结构 (Slotted-Page)	页头记录每个记录的位置和大小；可移动记录以保持连续 → 最常用

4.16.4 文件组织

组织方式	插入	删除	等值搜索	范围搜索	全扫描
堆文件 (Heap)	末尾追加 (2 I/O)	标记删除	0.5B (候选键) 或 B	B	B
顺序文件 (Sequential)	定位+溢出页 (B)	更新指针链 (B)	$\log_2 B$	$\log_2 B + \text{匹配页}$	B
哈希文件 (Hash)	哈希定位 (2 I/O)	哈希定位 (2 I/O)	1 I/O (无溢出)	B (不佳)	1.25B

堆文件：适合全扫描和批量操作；等值搜索需全扫描

顺序文件：记录按搜索键排序；适合范围查询和排序输出；需定期重组

哈希文件：对等值搜索最优 (1 次 I/O)；对范围搜索最差

4.16.5 NoSQL 数据分布

分片 (Sharding)

- 将大数据集划分为更小块分配到不同服务器
- 模块哈希：`server = hash(key) mod n`
- 问题：服务器数量变化时需要大量数据迁移

一致性哈希 (Consistent Hashing)

- 环形拓扑 `[0, 1]`，服务器和键都映射到环上
- 键存储在其顺时针方向第一台服务器上
- 添加/移除服务器只需移动约 `k/(n+1)` 个键

复制 (Replication)

- 每个物理服务器映射到环上多个位置 (虚拟节点/副本)
- 可扩展到多服务器冗余备份

4.17 Lecture 17: 索引 — 基础概念与有序索引

4.17.1 搜索键与索引

概念	说明
搜索键 (Search Key)	用于查找记录的属性（集）， 不一定是主键
索引文件 (Index File)	记录 <code><搜索键, 指针></code> 的文件，通常远小于数据文件
索引条目	通常 <code><搜索键值, 该记录/页的指针></code>

4.17.2 多级索引的效率

以 800 万条记录、每页 8 条记录、索引每页 100 条条目为例：

搜索方式	页访问成本
随机顺序 + 线性搜索	500,000（平均）
按搜索键排序 + 二分搜索	20
一级索引 + 二分搜索	15
二级索引	9
三级索引（根→叶→数据）	4

关键公式：树高 = $\lceil \log_{\text{fan-out}}(\text{叶子级别索引条目数}) \rceil$

4.17.3 有序索引 (Ordered Index)

每个节点中的索引条目按搜索键排序。搜索从根开始沿一条路径到达叶节点。

页 I/O 成本：树高（索引级别数） + 1（数据文件访问）

稠密 vs 稀疏索引

特性	稠密索引 (Dense)	稀疏索引 (Sparse)
索引条目	每个搜索键值一个条目	仅部分搜索键值有条目
空间开销	更大	更小

特性	稠密索引 (Dense)	稀疏索引 (Sparse)
维护开销	更大	更小
搜索方式	直接找到记录	找到页面后顺序搜索

主索引 vs 辅助索引

特性	主索引 / 聚簇索引 (Primary/Clustering)	辅助索引 / 非聚簇索引 (Secondary/Non-Clustering)
数据文件排序	按搜索键排序	不按搜索键排序
每个文件的索引数	最多 1 个	可以有多个
稠密/稀疏	通常是稀疏的	必须是稠密的 (数据不排序, 无法通过页面指针定位)

索引顺序文件 (ISAM): 有序、顺序文件 + 主索引

4.17.4 非候选键搜索键上的索引

策略	方法	问题
策略 1	变长条目 (一个键+多个指针)	实现复杂
策略 2	同一键多个条目	键冗余
策略 3 (最常用)	额外间接层: 索引条目→指针页→记录	通常仅增加 1 次页访问

→ 也称为**倒排文件 (Inverted File / Postings List)**

4.17.5 复合搜索键 (Composite Search Key)

- 多属性搜索键: 按**字典序**排列
- $(a_1, a_2) < (b_1, b_2)$ 当且仅当 $a_1 < b_1$ 或 $a_1 = b_1$ 且 $a_2 < b_2$

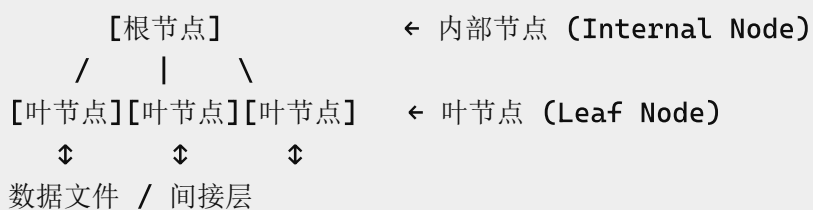
4.18 Lecture 18: 索引 — B+-树索引与哈希索引

4.18.1 B+-树 vs 有序索引

特性	有序索引 (ISAM)	B+-树
文件增长	性能退化（溢出页）	自动局部重组
重组	需定期全文件重组	不需要
平衡性	不保证	始终平衡（根→叶等长）
额外开销	低	插入/删除/空间开销

→ B+-树广泛用于所有商业 DBMS

4.18.2 B+-树结构与性质



节点容量（扇出 = n）：

节点类型	最少指针	最多指针	最少值	最多值
内部节点	$\lceil n/2 \rceil$	n	$\lceil n/2 \rceil - 1$	n-1
叶节点	$\lceil (n-1)/2 \rceil + 1$	n	$\lceil (n-1)/2 \rceil$	n-1
根（非叶）	2	n	1	n-1
根（叶）	0	n	0	n-1

B+-树阶 (Order)： $\lceil (n-1)/2 \rceil$ = 叶节点中的最小值数

内部节点指针规则

$P_1 \ K_1 \ P_2 \ K_2 \ \dots \ P_{n-1} \ K_{n-1} \ P_n$

- P_1 指向 $< K_1$ 的子树
- P_i 指向 $\geq K_{i-1}$ 且 $< K_i$ 的子树
- P_n 指向 $\geq K_{n-1}$ 的子树

叶节点

- 包含实际搜索键值 + 记录指针（或间接层指针）
- 最后指针 P_n 指向**右兄弟节点**（支持范围扫描）

4.18.3 B+-树四种类型

类型	数据文件排序	键类型	索引密度
聚簇、候选键（记录指针）	按搜索键	候选键	稠密
聚簇、候选键（页面指针）	按搜索键	候选键	稀疏
聚簇、非候选键	按搜索键	非候选键	稠密
非聚簇、候选键	不排序	候选键	稠密
非聚簇、非候选键	不排序	非候选键	稠密（+ 间接层）

4.18.4 B+-树查询

1. 从根开始
2. 找最小的 $K_i > v$ ，沿 P_i （左指针）向下
若无 $K_i > v$ ，沿 P_n （最后指针）向下
3. 重复直到叶节点
4. 在叶中找 $K_i = v$:
 - 找到 → 沿 P_i 到记录
 - 未找到 → 不存在

- 路径长度： $\leq \lceil \log_{\lceil n/2 \rceil}(K) \rceil$ （ K = 搜索键值数）
- 典型 $n \approx 100$ ，100 万搜索键值时最多 4 个节点

4.18.5 B+-树更新

插入

1. 找到应插入的叶节点 L
2. 若 L 有空间 → 插入
3. 若 L **溢出** → 分裂 (Split)
 - **Copy Up**: (叶节点) 前 $\lceil n/2 \rceil$ 值留在 L ，复制 $\lceil n/2 \rceil + 1$ 值并插入父节点
 - **Push Up**: (内部节点) $\lceil n/2 \rceil$ 位置的值被推到父节点
 - 分裂可递归到根 → 树增高

删除

1. 找到值所在叶节点 L，删除
2. 若 L **欠满** ($< \lceil (n-1)/2 \rceil$):
 - 先尝试从兄弟**重新分配**（借值）
 - 重新分配失败 → **合并** (Merge)
 - 合并可传播到根 → 树变矮

4.18.6 B+-树批量加载 (Bulk Loading)

- 比逐条插入高效得多
- 步骤：排序数据条目 → 按最小占用装载叶节点 → 自底向上构建内部节点
- 每个叶节点只写一次

4.18.7 哈希索引

特性	说明
组织方式	索引文件（非数据文件）是哈希文件
索引类型	始终是辅助索引
优点	等值搜索极快（1 I/O）
缺点	不支持范围搜索

静态哈希缺陷：

- 页数固定 → 数据库增长导致溢出页过多 → 性能退化
- 需要动态哈希技术

4.18.8 索引选择考量

因素	B+-树	哈希索引
等值搜索	✓ 高效	✓ 最高效
范围搜索	✓ 高效	✗ 不支持
使用频率	~90% 查询	主要用于主存哈希
更新开销	中等	低

4.19 Lecture 19: 查询处理 — 成本估算

4.19.1 查询处理概览

SQL 查询 → 解析器 (Parser) → 关系代数表达式

↓

查询结果 ← 求值引擎 (Evaluation) ← 优化器 (Optimizer) → 执行计划

- 同一查询可能有**多个等价的关系代数表达式**
- 每个操作可用**不同算法**实现
- 优化器选择**预估成本最低**的执行计划

4.19.2 成本度量

- 以**页 I/O 次数**度量成本
- 忽略 CPU 成本和顺序/随机 I/O 差异 (简化假设)
- **不包含**最终输出写入辅存储的成本
- 使用数据库目录中的统计信息

4.19.3 选择操作 — 等值搜索

算法	适用条件	候选键成本	非候选键成本
A1: 线性搜索	总是可用	$B/2$	B
A2: 二分搜索	文件按搜索键排序	$\lceil \log_2 B \rceil$	$\lceil \log_2 B \rceil + \text{匹配页数}$
A3: 聚簇索引	B+-tree 聚簇索引	$HT_i + 1$	$HT_i + \text{匹配页数}$
A4: 非聚簇索引	B+-tree 辅助索引	$HT_i + 1$	$HT_i + \text{间接指针} + \text{记录数}$

哈希索引 (辅助索引, 候选键): $1 + 1$ (或 $1.2 + 1$ 若有溢出)

4.19.4 选择操作 — 范围搜索

算法	条件
A5: 聚簇 B+-tree	$\sigma_{A \geq v}$: 用索引找首记录, 顺序扫描; $\sigma_{A \leq v}$: 不用索引, 顺序扫描到 $A > v$
A6: 非聚簇 B+-tree	$\sigma_{A \geq v}$: 扫描索引叶页找指针; $\sigma_{A \leq v}$: 扫描索引叶页直到 $A > v$

4.19.5 选择操作 — 复杂谓词

算法	谓词类型	方法
A7	合取 (AND) 单索引	选成本最低的 q_i 用索引，在缓冲中检查其余条件
A8	合取 复合索引	使用复合（多属性）索引
A9	合取 指针求交	多个索引扫描 + 指针交集 + 取记录
A10	析取 (OR) 指针求并	所有条件都需索引；指针并集 + 取记录

否定 (NOT): 使用线性搜索；若否定条件选择性很高且有索引，可间接使用索引。

4.19.6 外部排序 (External Sort-Merge)

当数据太大无法全部装入主存时使用。

算法:

1. **Pass 0 (创建排好序的 Run):** 每 M 页读入缓冲，排序，写回 → 创建 B/M 个 run
2. **合并 Pass:** 合并 $M-1$ 个 run，直到所有 run 合并为一个

成本:

- Pass 数: $1 + \lceil \log_{M-1}(B/M) \rceil$
- I/O 成本: $2B \times (1 + \lceil \log_{M-1}(B/M) \rceil)$

两次完成排序所需缓冲页数: $M \approx \sqrt{B}$

4.20 Lecture 20: 查询处理 — 连接操作与表达式求值

4.20.1 投影操作

场景	方法
无重复消除	生成结果元组时移除不需要的属性 → 无额外成本
需要 DISTINCT	修改的外部排序（排序时移除不需要属性 + 合并时消除重复）
索引可用	若稠密索引包含所有需要的属性 → 仅索引扫描

4.20.2 连接操作 (JOIN)

参考符号：r（外关系），s（内关系）， n_r （r 的元组数）， B_r （r 的页数），M（缓冲页数）

块嵌套循环连接 (Block Nested-Loop Join)

条件	成本公式
最坏情况	$B_r \times B_s + B_r$
最好情况	$B_r + B_s$
优化（M-2 页阻塞）	$\lceil B_r / (M-2) \rceil \times B_s + B_r$

- 若某关系能放入缓冲 → 用它作**内关系 s**
- 若都不能放入 → 用**较小关系作外关系 r**
- 若连接属性是内关系的主键 → 可在**第一个匹配后停止**内循环

索引嵌套循环连接 (Indexed Nested-Loop Join)

成本： $B_r + n_r \times c$

- c = 索引搜索 + 取匹配 s 元组的成本
- **最佳场景：** 外关系选择性高（少量元组满足条件）
- 若两边都有索引 → 用较小的关系作外关系

归并连接 (Merge-Join)

成本： $B_r + B_s$ （已排序）或 $B_r + B_s + \text{排序成本}$ （未排序）

- 仅适用于等值连接和自然连接
- 每页只读一次（假设同值元组都在缓冲中）

哈希连接 (Hash-Join)

成本： $3 \times (B_r + B_s)$

阶段	I/O
创建分区	1 读 + 1 写（两个关系各一次）
计算连接	1 读（两个关系各一次）

算法:

1. 用哈希函数 h 将 r 和 s 分区为 n 个桶
 - 只比较分区 r_i 和 s_i (同哈希值的元组)
2. 对每个 i : 将 r_i 载入缓冲 $\rightarrow$ 建主存哈希索引 $\rightarrow$ 逐页探测 s_i

约束:

- n 个分区: 每个 build input 分区必须 $\leq M$ 页
- $M \geq n+1$ (每个分区一个缓冲页 + 一个输入页)
- build input 应选**较小关系**

4.20.3 复杂连接

条件	方法
合取条件	计算最选择性连接的 join, 在缓冲中检查其余条件
析取条件	各简单连接的 join 结果取并集 (仅在所有条件都选择性高时有用)

4.20.4 表达式求值

物化 (Materialization)

- 一次求值一个操作, 结果写入临时关系
- **总是可用**, 但中间结果的读写成本高

流水线 (Pipelining)

- 多个操作同时求值, 结果直接传递
- **成本远低于物化** (无需写入辅存储)
- **需求驱动 (惰性)**: 顶层请求元组, 逐层下推
- **生产者驱动 (急切)**: 持续产生元组, 放入缓冲
- **阻塞操作**: 归并连接、哈希连接 (需等全部输入才能输出)

4.21 Lecture 21: 查询处理 — 大小估算与查询优化

4.21.1 数据库统计信息

统计量	含义
n_r	r 的元组数
B_r	r 的页数
l_r	r 的元组大小 (字节)
bf_r	r 的阻塞因子
$V(A, r)$	A 在 r 中不同值数量
HT_i	索引 i 的高度
LB_i	索引 i 底层页数

4.21.2 选择基数与选择性

概念	公式
选择基数 $SC(q, r)$	满足谓词 q 的平均元组数
选择性 $Selectivity(q, r)$	$SC(q, r) / n_r$ (0 到 1 之间)

等值选择 (非候选键): $SC(A=v, r) = n_r / V(A, r)$

等值选择 (候选键): $SC(A=v, r) = 1$

范围选择: $SC(A < v, r) = n_r \times (v - \min(A, r)) / (\max(A, r) - \min(A, r))$

4.21.3 直方图 (Histogram)

- 解决**均匀分布假设**不准确的问题
- **等宽直方图**: 按值的范围等分
- **等深直方图**: 按元组数等分

4.21.4 复杂选择的大小估算

合取: $SC(\sigma_{q_1 \wedge q_2}, r) = n_r \times ($

$$s_1 \times s_2$$

$)$ (假设属性独立)

析取: $SC(\sigma_{q_1 \vee q_2}, r) = n_r \times (1 - (1-s_1)(1-s_2))$

4.21.5 连接大小估算

条件	估算结果大小
$r \cap s = \emptyset$	$n_r \times n_s$ (笛卡尔积)
$r \cap s$ 是 r 的候选键	$\leq n_s$
$r \cap s$ 是 s 中引用 r 的外键	$= n_s$
$r \cap s$ 不是任一方键	取 $\min(n_r \times n_s / V(A,s), n_s \times n_r / V(A,r))$

4.21.6 其他操作大小估算

操作	估算大小
投影	$V(A, r)$
分组	$V(A, r)$
并	$\text{size}(r) + \text{size}(s)$
交	$\min(\text{size}(r), \text{size}(s))$
差	$\text{size}(r)$

4.21.7 查询优化

基于成本的优化 (Cost-Based)

- 1. 使用等价规则生成逻辑等价求值计划
- 2. 估算每个计划的成本
- 3. 执行成本最小的计划

启发式优化 (Heuristic)

- 1. 尽早执行选择 (减少元组数)
- 2. 尽早执行投影 (减少元组大小)
- 3. 利用现有索引
- 4. 先执行最具选择性的操作

4.21.8 关系代数等价规则

规则	说明
选择级联	$\sigma_{c_1 \wedge c_2}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(R))$

规则	说明
选择交换	$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$
投影级联	$\pi_{a_1}(R) \equiv \pi_{a_1}(\pi_{a_2}(R)) \quad (a_1 \subseteq a_2)$
连接交换	$R \bowtie S \equiv S \bowtie R$
连接结合	$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
选择+连接交换	$\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S \quad (c \text{ 仅涉及 } R)$
投影+连接分配	$\pi_a(R \bowtie S) \equiv \pi_a(\pi_{a_1}(R) \bowtie \pi_{a_2}(S))$

4.22 Lecture 22: 事务管理 — 事务与可串行化

4.22.1 事务与 ACID 属性

属性	含义	处理机制
Atomicity (原子性)	全部执行 或 全部不执行	恢复系统
Consistency (一致性)	事务隔离执行保持数据库一致	并发控制
Isolation (隔离性)	并发事务互相不可见中间结果	并发控制
Durability (持久性)	提交后的更改不丢失	恢复系统

4.22.2 事务状态转换

Active → Partially Committed → Committed
 ↓
 Failed → Aborted (→ 可选择重启或终止)

4.22.3 调度的正确性

调度类型	说明
串行调度 (Serial)	一次只执行一个事务 (无并发)
可串行化调度 (Serializable)	并发执行效果等价于某串行调度
冲突可串行化	可通过交换非冲突指令变成串行调度

4.22.4 冲突 (Conflict)

操作组合	是否冲突
read(Q) + read(Q)	不冲突
read(Q) + write(Q)	冲突
write(Q) + read(Q)	冲突
write(Q) + write(Q)	冲突

→ 如果两个指令连续且不冲突，**交换它们不改变结果**

4.22.5 优先图 (Precedence Graph)

- 顶点 = 事务
- 边 $T_i \rightarrow T_j = T_i$ 比 T_j 先访问冲突的数据项
- **调度冲突可串行化 $\Leftrightarrow$ 优先图无循环**

4.22.6 可恢复性 (Recoverability)

调度类型	条件
可恢复 (Recoverable)	T_j 读取 T_i 写的数据后， T_j 在 T_i 之后提交
无级联回滚 (Cascadeless)	T_j 只能在 T_i 提交后 才能读取 T_i 写的数据
关系	级联无级联 $\subset$ 可恢复

→ 调度必须**可恢复**，且最好**级联无级联**

4.23 Lecture 23: 事务管理 — 并发控制协议

4.23.1 锁 (Lock)

锁模式	可执行操作	请求指令
共享锁 (Shared/S)	只能读	lock-s(Q)
排他锁 (Exclusive/X)	可读可写	lock-x(Q)
解锁	释放锁	unlock(Q)

锁兼容矩阵

持有\请求	S	X
S	✓ Granted	X Denied
X	X Denied	X Denied

→ 任意数量的事务可共享 S 锁；若任何事务持有 X 锁，其他事务都不能获锁。

4.23.2 两阶段锁协议 (Two-Phase Locking, 2PL)

阶段	允许操作
Growing Phase (增长阶段)	获取或升级锁
Shrinking Phase (收缩阶段)	释放或降级锁

- 2PL → 调度必为冲突可串行化
- 事务可按锁点（获得最后锁的时刻）排序
- ⚠️ 不是所有冲突可串行化的调度都被 2PL 允许

4.23.3 2PL 可恢复性改进

变体	规则	保证
严格 2PL (Strict 2PL)	所有排他锁保持到事务提交	级联无级联 + 可恢复
严格 2PL (Rigorous 2PL)	所有锁保持到事务提交	级联无级联 + 可恢复

4.23.4 死锁 (Deadlock)

两个事务互相等待对方释放锁 → 必须回滚其中一个。

死锁处理

方法	策略
死锁预防	

方法	策略
— 排序锁请求	所有事务按相同（部分/全）序申请锁
— Wait-Die	老事务等待年轻的；年轻事务回滚（"die"）
— Wound-Wait	年轻事务等待老的；老事务"伤害"（回滚）年轻事务
死锁检测	使用等待图 (Wait-for Graph)；有循环 = 死锁
死锁恢复	选择牺牲品（最小成本）→ 事务回滚

等待图：顶点 = 事务，边 $T_i \rightarrow T_j = T_i$ 等待 T_j 释放数据项。

4.23.5 锁管理器实现

- 锁管理器作为独立进程运行
- 维护**锁表**（主存哈希表，按数据项索引）
- 锁请求按 FIFO 顺序排队和授权
- 解锁导致检查队列中下一个请求是否可授权

4.23.6 时间戳协议 (Timestamp-Ordering Protocol)

每个事务 T_i 在开始前获得固定时间戳 $TS(T_i)$ 。时间戳决定可串行化顺序。

每个数据项 Q 维护：

- **RTS(Q)**：成功执行 $read(Q)$ 的最大时间戳
- **WTS(Q)**：成功执行 $write(Q)$ 的最大时间戳

读操作规则

条件	动作
$TS(T_i) < WTS(Q)$	回滚 T_i （试图读已被新事务覆盖的值）
$TS(T_i) \geq WTS(Q)$	执行读， $RTS(Q) = \max(RTS(Q), TS(T_i))$

写操作规则

条件	动作
$TS(T_i) < RTS(Q)$	回滚 T_i （新事务已读旧值）

条件	动作
$TS(T_i) < WTS(Q)$	回滚 T_i (新事务已写新值)
否则	执行写, $WTS(Q) = TS(T_i)$

Thomas 写规则 (Thomas' Write Rule)

原规则	Thomas 规则
$TS(T_i) < WTS(Q) \rightarrow$ 回滚	$TS(T_i) < WTS(Q) \rightarrow$ 忽略此写

→ 如果新事务已写了更新的值, 忽略旧值写入 (更高效, 避免不必要回滚)

4.23.7 多版本并发控制 (Multiversion)

- 保持数据项的多个版本 (标记时间戳)
- 每个版本 Q_k 包含: Content, $WTS(Q_k)$, $RTS(Q_k)$

多版本时间戳排序读:

- 读总是成功 → 返回 $WTS \leq TS(T_i)$ 的最大版本的内容
- 读**从不失败、从不等待**

多版本时间戳排序写:

- $TS(T_i) < RTS(Q_k) \rightarrow$ 回滚
- $TS(T_i) = WTS(Q_k) \rightarrow$ 覆盖内容
- $TS(T_i) > WTS(Q_k) \rightarrow$ 创建新版本

4.24 Lecture 24: 事务管理 — 恢复系统与 NoSQL 事务

4.24.1 DBMS 故障分类

故障类型	说明
事务故障 (Transaction Failure)	
— 逻辑错误	事务因内部错误无法完成 (如逻辑错误、无效输入)

故障类型	说明
— 系统错误	DBMS 必须终止活动事务（如死锁）
系统崩溃 (System Crash)	因电源、硬件或软件故障导致；假设非易失性存储内容不被破坏（Fail-stop 假设）
辅存储故障	磁盘故障破坏部分或全部磁盘存储；假设可被检测（校验和）

4.24.2 恢复算法

恢复算法有两个部分：

1. **正常事务处理期间**采取的操作 — 确保有足够信息用于恢复
2. **故障发生后**的操作 — 将数据库恢复到确保原子性、持久性和一致性的状态

目标： 尽管发生故障，仍确保事务原子性、持久性和数据库一致性。

4.24.3 数据访问假设

概念	说明
物理页 (Physical Pages)	驻留在辅存储上的数据库页面
缓冲页 (Buffer Pages)	临时驻留在主存中的数据库页面
input(B)	将物理页 B 从辅存储传入缓冲区
output(B)	将缓冲页 B 替换到辅存储
read(x)	将数据库项 x 的值赋给局部变量 x_i （可能需要 input）
write(x)	将局部变量 x_i 的值赋给缓冲区中的 x（不需要立即 output）

- 每个事务有**私有工作区**（局部副本）
- 事务在**首次访问** x 时执行 read(x)，后续访问操作局部副本 x_i
- 在**最后一次访问** x_i 后执行 write(x)
- `output(BX)` **不需要**立即跟随 `write(x)`

4.24.4 基于日志的恢复 (Log-Based Recovery)

日志 (Log)： 记录所有数据库更新活动的记录序列，保存在**稳定存储**（如磁盘）上。

日志记录	写入时机	内容
<code><T_i start></code>	事务 T _i 开始时	注册事务
<code><T_i, x, \$v_1\$, \$v_2\$></code>	T _i 执行 write(x) 之前	v_1 =旧值, v_2 =新值
<code><T_i commit></code>	T _i 完成最后一条指令时	标记提交

延迟数据库修改 (Deferred Modification)

所有修改**记录到日志**，但写入数据库**推迟到部分提交后**。

- 日志记录仅含**新值 v**： `<Ti, x, v>`
- 当 T_i 部分提交时，写入 `<Ti commit>`
- 最后读取日志记录并执行被推迟的写入

恢复规则：事务需要 **redo** ⇔ 日志中同时有 `<Ti start>` 和 `<Ti commit>`

- `redo(Ti)`：将所有被 T_i 更新的数据库项设为其新值
- 既无 start 也无 commit → **忽略**，重启事务

立即数据库修改 (Immediate Modification)

未提交事务的数据库更新**可以在 write 发出时立即执行**。

- 日志必须同时包含**旧值 v_1 和新值 v_2** ： `<Ti, x, $v_1$, $v_2$>`
- 更新日志记录必须在数据库项被写入**之前**写入
- 更新页的 output 可在事务提交**之前或之后**进行
- output 的顺序可能与写入缓冲区的顺序**不同**

恢复规则：

- `undo(Ti)`：从最后一条记录向后，将 T_i 更新过的所有项恢复为**旧值**
- `redo(Ti)`：从第一条记录向前，将 T_i 更新过的所有项设为**新值**
- 两种操作必须是**幂等的 (idempotent)**
- T_i 需要 undo ⇔ 日志中有 `<Ti start>` 但无 `<Ti commit>`
- T_i 需要 redo ⇔ 日志中同时有 `<Ti start>` 和 `<Ti commit>`
- **undo 先执行，redo 后执行**

4.24.5 检查点 (Checkpoints)

问题：

1. 搜索整个日志文件**耗时**
2. 可能**不必要地 redo** 已输出更新的事务

检查点操作：

1. 输出缓冲中所有日志记录到稳定存储
2. 输出所有已修改的缓冲页到辅存储
3. 写入 `<checkpoint>` 日志记录

事务在检查点执行期间**不能执行任何更新操作**。

串行执行下的恢复（含检查点）

1. 从日志末尾**向后扫描**找到最近的 `<checkpoint>`
2. 继续向后扫描直到找到 `<Ti start>` 记录
3. 只需考虑此 start 记录**之后**的日志部分；之前的部分可忽略
4. **向前扫描**日志（从 T_i 开始）：
 - 无 `<Ti commit>` → 执行 `undo(Ti)`（仅在立即修改时）
 - 有 `<Ti commit>` → 执行 `redo(Ti)`

示例：

	checkpoint	failure	
T ₁ :			← 可忽略（检查点前已提交）
T ₂ :			← 需要 redo（有 commit）
T ₃ :			← 需要 redo（有 commit）
T ₄ :			← 需要 undo（无 commit）

4.24.6 并发事务的恢复

当允许多事务并发执行时（使用 **strict 2PL**）：

- 所有事务共享**单一缓冲区和单一日志**
- 不同事务的日志记录可能在日志中**交错**
- 一个缓冲页可能包含**多个事务**更新的数据库项

检查点格式变更

`<checkpoint L>` — L 是检查点时**活跃事务**（未提交）的列表。

并发恢复算法

第一阶段 — 构建列表：

1. 初始化 undo-list 和 redo-list 为空
2. 从日志末尾**向后扫描**，到第一个 `<checkpoint L>` 记录停止
3. 对每个记录：
 - 若是 `<Ti commit>` → T_i 加入 **redo-list**
 - 若是 `<Ti start>` 且 T_i 不在 redo-list → T_i 加入 **undo-list**
4. 对 L 中每个 T_i，若不在 redo-list → 加入 **undo-list**

第二阶段 — 执行恢复：

1. **向后扫描**日志（从最后记录到所有 undo-list 事务的 start）→ 执行 **undo**
2. **向前扫描**日志（从 checkpoint 到末尾）→ 对 redo-list 执行 **redo**

| 向后 undo 恢复原始值；向前 redo 设置每项为最新值。

4.24.7 写前日志 (Write-Ahead Logging, WAL)

若日志记录**缓冲在主存中**（非直接输出），须遵守 WAL 规则：

规则	说明
1. 顺序输出	日志记录按 创建顺序 输出到稳定存储
2. 提交前输出	T _i 进入提交状态仅在 <code><T_i commit></code> 输出到稳定存储 之后
3. 数据输出前先输出日志	缓冲页输出到数据库 之前 ，该页相关的所有日志记录必须已输出

| **Log Force**：通过强制将所有日志记录（含 commit 记录）输出到稳定存储来提交事务。

4.24.8 NoSQL 分布式事务

本地 vs 全局事务

类型	说明
本地事务 (Local)	仅访问和更新 一个节点 上的数据

类型	说明
全局事务 (Global)	访问和更新多个节点上的数据

事务协调架构

角色	职责
事务协调器 (Transaction Coordinator)	启动事务、分发子事务到适当站点、协调终止
本地事务管理器 (Local Transaction Manager)	维护日志用于恢复、协调该站点事务的提交/中止

提交协议：确保原子性 — 多站点事务必须在所有站点都提交或都中止。

4.24.9 CAP 定理

分布式数据库系统不能同时保证以下三个属性：

属性	含义
Consistency（一致性）	所有节点同时看到相同数据
Availability（可用性）	每个请求都能收到成功/失败的响应
Partition Tolerance（分区容忍）	即使部分节点宕机，系统继续工作

在网络分区存在时，必须在一致性和可用性之间选择：

- 关系型 DBMS → 选择一致性（一致性优先）
- NoSQL DBMS → 选择可用性（可用性优先）

4.24.10 BASE 原则（最终一致性）

NoSQL DBMS 遵循 BASE 原则而非 ACID：

属性	说明
Basically Available（基本可用）	遵守 CAP 定理的可用性保证
Soft state（软状态）	系统可随时间变化，即使没有输入（因网络延迟更新）
Eventually consistent（最终一致）	系统随时间最终将变得一致

4.24.11 事务管理总结

方面	关系型 DBMS (ACID)	NoSQL DBMS (BASE)
核心保证	强一致性	最终一致性
并发控制	锁协议 (2PL) / 时间戳协议	更宽松
恢复机制	日志 + WAL + 检查点	复制 + 最终一致
分布式事务	提交协议 (原子跨站点)	CAP 保证: 可用性 > 一致性
适用场景	银行、金融	社交网络、Web 应用

4.24.12 Part 4 核心知识地图

存储与文件结构 (L16)

- 存储层次: 主存(易失) → SSD/HDD(持久) → 磁带(归档)
- 数据库缓冲: 缓冲区管理器, LRU/MRU 替换策略
- 记录组织: 定长(空闲链表) vs 变长(槽页结构最常用)
- 文件组织对比: 堆(全扫描) vs 顺序(范围查询) vs 哈希(等值查询)
- NoSQL 分布: 分片, 一致性哈希(环), 复制

索引基础 (L17)

- 搜索键 ≠ 主键(可为任意属性)
- 多级索引: 树高 = $\log(\text{扇出})$ (条目数)
- 稠密 vs 稀疏: 每个值 vs 部分值
- 主/聚簇索引 vs 辅助/非聚簇索引: 排序 vs 不排序
- 非候选键索引: 间接层/倒排文件
- 复合搜索键: 字典序排列

B+-树索引 (L18)

- 结构: 平衡树, 根→叶等长
- 节点: 内部节点($\lceil n/2 \rceil \sim n$ 指针) + 叶节点
- 查询: 对数复杂度($\sim \log_{\text{扇出}}(K)$)
- 更新: 溢出→分裂(Copy/Push Up); 欠满→重新分配/合并
- 批量加载: 排序+自底向上(效率高)
- 四种形态: 聚簇/非聚簇 × 候选键/非候选键
- 哈希索引: 等值快, 不支持范围, 静态有缺陷

查询处理 (L19-L21)

- 成本度量: 页I/O, 忽略CPU/顺序vs随机
- 选择算法: 线性/二分/聚簇索引/辅助索引
- 外部排序: Pass数 = $1 + \lceil \log_{M-1}(B/M) \rceil$, 两遍需 $M \approx \sqrt{B}$
- 连接算法: 块嵌套循环/索引嵌套循环/归并/哈希连接
- 表达式求值: 物化 vs 流水线
- 大小估算: 基数SC, 选择性selectivity, 直方图
- 等价规则: 选择级联/交换, 连接交换/结合, 选择和投影下推
- 优化: 成本-based + 启发式(早选择/早投影/用索引)

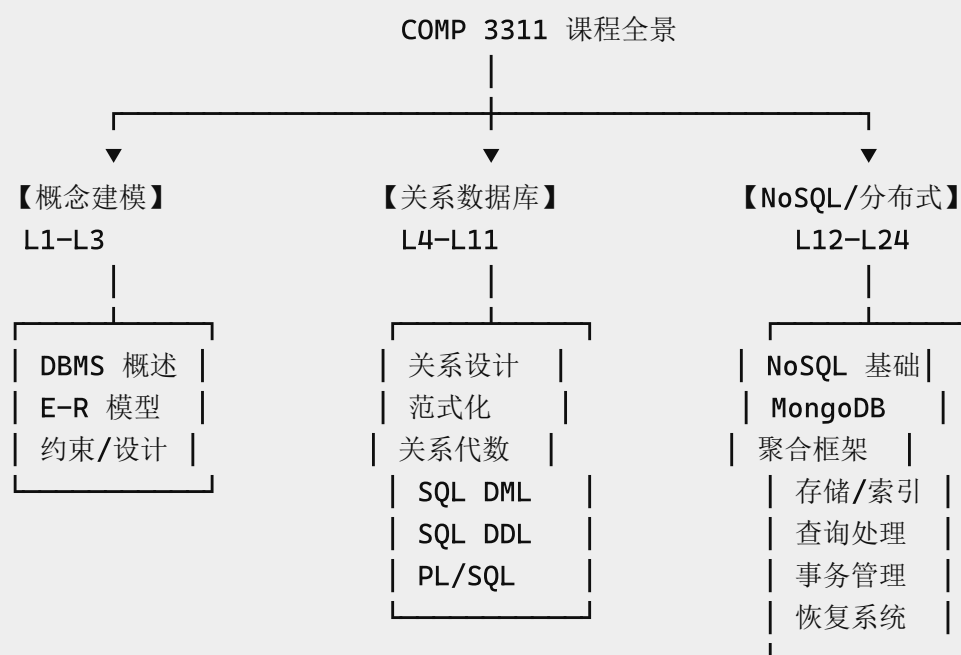
事务管理：并发控制（L22-L23）

- └─ ACID：原子性(恢复) + 一致性(并发控制) + 隔离性(并发控制) + 持久性(恢复)
- └─ 调度：串行 → 可串行化 → 冲突可串行化(优先图无环)
- └─ 可恢复性：可恢复 → 级联无级联
- └─ 锁：共享(S)/排他(X)，锁兼容矩阵
- └─ 2PL：Growing→Shrinking，保证冲突可串行化
- └─ 严格2PL：排他锁(严格) 或 所有锁(Rigorous) 保持到提交
- └─ 死锁：预防(排序/Wait-Die/Wound-Wait) + 检测(等待图)
- └─ 时间戳协议：RTS/WTS 规则 + Thomas写规则
- └─ 多版本：保留旧版本，读永不失败，版本按时间戳管理

事务管理：恢复系统（L24）

- └─ 故障分类：事务故障(逻辑/系统) + 系统崩溃 + 辅存储故障
- └─ 数据访问：read(x)/write(x)，input/output，私有工作区，缓冲页
- └─ 日志基础：<Ti start> + <Ti, x, v1, v2> + <Ti commit>
- └─ 延迟修改：仅存储新值，提交后写入，恢复仅做 redo
- └─ 立即修改：存储新旧值，立即写入，恢复做 undo + redo
- └─ 检查点：输出日志+缓冲页 → <checkpoint>，限制扫描范围
- └─ 并发恢复：<checkpoint L> 含活跃事务列表，undo-list + redo-list
- └─ WAL规则：日志顺序输出，commit先于数据，数据输出前日志已输出
- └─ NoSQL事务：本地(单节点) vs 全局(多节点)，协调器+本地管理器
- └─ CAP定理：一致性/可用性/分区容忍 三者不可兼得
- └─ BASE：基本可用 + 软状态 + 最终一致性 (vs ACID)

课程知识体系总图



参考模式速查

Bank Schema (SQL)

```
Account(accountNo, balance, branchName)
Borrower(clientId, loanNo)
Branch(branchName, district, liabilities, assets)
Client(clientId, name, hkid, address, district, rating)
Depositor(clientId, accountNo)
Loan(loanNo, amount, year, branchName)
Tags(clientId, tag)
```

Bank Schema (MongoDB)

```
clients: {clientId, name, hkid, address, district, rating,
          accounts: [{accountNo, balance, branch}],
          loans: [{loanNo, amount, year, branch}],
          tags: [...]}
branches: {branch, district, liabilities, assets,
           accounts: [...], loans: [...]}
```

University Schema

```
Student(studentId, name, year, deptName)
Course(courseNo, title, credits, deptName)
Instructor(instId, name, deptName, salary)
Enrol(studentId, courseNo, semester, grade)
Teaching(instId, courseNo, semester, room)
Department(deptName, building, budget)
```

Reviewers/Submitters Schema (MongoDB)

```
submitters: {sid, name, email, proposals: [{pid, title, area}, ...]}
reviewers: {rid, name, email, expertise, reviews: [{pid, score}, ...]}
```

MongoDB 聚合框架 — 模式速查

阶段	功能	SQL 对应
\$project	限制/变换字段	SELECT
\$match	过滤文档	WHERE/HAVING
\$group	分组聚合	GROUP BY
\$count	计数	COUNT
\$sort	排序	ORDER BY
\$limit	限制行数	FETCH
\$skip	跳过	OFFSET
\$unwind	数组展开	N/A
\$lookup	左外连接	LEFT JOIN

常见流水线模式：

展开+过滤： \$unwind → \$match
展开+聚合： \$unwind → \$group(累加器)
分组+排序： \$group → \$sort
连接+过滤： \$lookup → \$unwind → \$match
去重+求和： \$unwind → \$group(\$addToSet) → \$unwind → \$group(\$sum)
Top-N(含并列)： \$group(\$push) → \$sort → \$skip → \$limit

课程: COMP 3311 Database Management Systems — Summer 2026

教材: "Fundamentals of Database Systems"

完整笔记: 覆盖 Lecture 1–24 全部内容，共 4 部分、9 个主题模块

Dataview (inline field '等于 > 大于 < 小于

>= 大于等于 <= 小于等于 <> 或 != 不等于'): Error:

-- PARSING FAILED -----

1 | 等于 > 大于 < 小于
> 2 | >= 大于等于 <= 小于等于 <> 或 != 不等于
| ^

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable

Dataview (for inline query '= SOME(set)'): Unrecognized function name 'SOME'

Dataview (for inline query '= ALL(set)'): Unrecognized function name 'ALL'